

Building makemore Part 3： Activations & Gradients, BatchNorm

从零构建 makemore 第三部分：激活、梯度与批量归一化（字幕版）｜深入理解神经网络训练中的数值稳定性

时长 1:55:57 | 句数 1223 | 术语 58 | 章节 16

摘要

本讲共 16 章，从「多层感知器 · 多层感知机」到「反向传播」。全文 1223 句、1223 句已译成中文，标注了 59 个 AI 术语（主要集中在神经网络基础、数学与微积分、训练与优化）。

16 chapters, 1223 sentences (1223 translated), 59 AI terms annotated. Built fully offline with opus-mt + local glossary.

多层感知器 · 多层感知机 00:00

00:00 Hi everyone. Today we are continuing our implementation of Make More.
大家好,今天我们继续执行《创造更多》。

implementation
实现 实现
代码与工程

00:04 Now in the last lecture we implemented the multilayer perceptron along the lines of
在最后的演讲中,我们按照

multilayer perceptron
多层感知机 多层感知机
神经网络基础

00:07 Benjio et al 2003 for character level language modeling.
Benjio等人 2003年,用于品格级语言模型。

00:10 So we followed this paper, took in a few characters in the past,
所以我们跟着这篇论文,过去录了一些人物

00:14 and used an MLP to predict the next character in a sequence.
并使用 MLP 来预测序列中的下一个字符。

mlp
多层感知器 多层感知器
神经网络基础

00:17 So what we'd like to do now is we'd like to move on to more complex and larger
所以我们现在想做的是 我们想要更复杂,更大的

00:20 neural networks, like recurrent neural networks and their variations like the
神经网络,如循环神经网络 和它们的变异,如

00:24 GRU, LSTM, and so on.
古鲁,LSTM等等。

00:26 Now before we do that though, we have to stick around the level of
现在在我们这样做之前,我们必须保持

00:29 multilayer perceptron for a bit longer.
多层透视器再长一点。

multilayer perceptron
多层感知机 多层感知机
神经网络基础

00:31 And I'd like to do this because I would like us to have a very good intuitive
我想这样做,因为我希望我们有一个非常 良好的直觉

00:34 understanding of the activations in the neural net during training,
了解培训期间神经网中的激活,

neural net
神经网络 神经网络
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

00:38 and especially the gradients that are flowing backwards,
特别是向后流的梯度,

00:40 and how they behave, and what they look like.
他们的行为方式,和他们的样子。

00:43 And this is going to be very important to understand the history of the
这一点非常重要, 了解历史

00:45 development of these architectures, because we'll see that recurrent neural
因为我们可以看到这些反复出现的神经

00:49 networks, while they are very expressive, in that they are a universal approximator
网络虽然非常直观,但是一个普遍接近者

00:53 and can in principle implement all the algorithms,
原则上可以实施所有算法

00:57 we'll see that they are not very easily optimizable with the first order gradient
以第一阶梯度来优化它们,我们就会发现

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

01:01 based techniques that we have available to us and that we use all the time.
基础技术,我们拥有的 我们可以利用的,我们使用所有的时间。

01:04 And the key to understanding why they are not optimizable easily
以及理解为什么它们不易达到最佳程度的关键

01:08 is to understand the activations and the gradients and how they behave during
是要理解激活和梯度,以及它们如何在

01:11 training. And we'll see that a lot of the variants since recurrent neural
训练后,我们会看到很多变异体 自神经循环

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

01:15 networks have tried to improve that situation.
网络试图改善这种状况。

01:19 And so that's the path that we have to take and let's go start it.
所以我们必须走这条路,让我们开始吧。

01:22 So the starting code for this lecture is largely the code from before,
所以这次讲座的开始代码 主要是以前写的代码

01:26 but I've cleaned it up a little bit. So you'll see that we are importing
但我清理了一下,所以你可以看到我们进口

01:30 all the torch and math plotlet utilities. We're reading in the words just like before.
所有的火炬和数学图纸都用词 我们读的字像以前一样

01:35 These are eight example words. There's a total of 32,000 of them.
这八个字是八个例子,总共32 000个

01:39 Here's a vocabulary of all the lowercase letters and the special dot token.
这是所有小写字母和特殊点符号的词汇表

01:44 Here we are reading the data set and processing it and creating three splits,
我们在这里阅读和处理数据组 并分成三部分

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

01:50 the train, dev and the test split. Now in the MLP, this is the identical same MLP,
列车,德夫和测试分开。现在在 MLP,这是相同的 MLP

mlp
多层感知器 多层感知器
神经网络基础

01:56 except you see that I removed a bunch of magic numbers that we had here.
除了你看,我删除了一堆 神奇的数字,我们在这里。

01:59 And instead we have the dimensionality of the embedding space of the characters
而不是我们拥有了 角色的嵌入空间的维度

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

02:03 and the number of hidden units in the hidden layer. And so I've pulled them outside here
隐藏层中隐藏单位的数量。 所以我把它们拉到外面

hidden layer
隐藏层 神经网络 中的合成层，介于输入层 （即特征）和输出层 （即预测）之间。神经网络包含一个或多个隐藏层。
神经网络基础

02:07 so that we don't have to go and change all these magic numbers all the time.
这样我们不必去 改变所有这些神奇的数字

02:11 With the same neural net with 11,000 parameters that we optimize now over 200,000
同样的神经网与11 000个参数,我们现在优化超过20万

neural net
神经网络 神经网络
神经网络基础

02:15 steps with a batch size of 32. And you'll see that I refactored the code here a little bit,
相形之下,32个级数的体积。 你会看到我在这里重新输入了代码一点点,

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

02:22 but there are no functional changes. I just created a few extra variables,
但功能没有变化。我只是创造了一些额外的变量,

02:25 a few more comments, and I removed all the magic numbers and otherwise it's the exact same thing.
再说几句,我删除了所有的神奇数字 否则就完全一样

02:31 Then when we optimize, we saw that our loss looked something like this.
当我们优化时,我们看到我们的损失看起来是这样的。

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

02:35 We saw that the train and val loss were about 2.16 and so on.
我们看到火车和Val损失大约2.16,等等。

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

02:41 Here I refactored the code a little bit for the evaluation of arbitrary splits.
我在这里重新定义了代码 来评估任意分裂

02:46 So you pass in a string of which split you'd like to evaluate.
所以,你通过一个字符串 分裂的你想要评估。

02:49 And then here, depending on train, val or test, I index in and I get the correct split.
然后在这里,取决于火车, Val或测试, 我输入我指数,我得到正确的分数。

02:55 And then this is the forward pass of the network and evaluation of the loss and printing it.
然后这是网络的远端传承 以及损失和打印的评估

forward pass
前向传播 前向传播
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

损失函数 · 概率 02:59

02:59 So just making it nicer. One thing that you'll notice here is
所以,你只要把它更好。

03:05 I'm using a decorator torch.no grad, which you can also look up and read documentation of.
我用的是一台装饰手电筒, 诺研究生, 你可以查查看文件。

03:11 Basically what this decorator does on top of a function is that whatever happens in this function
基本上这个设计器在函数上部所做的 是一个函数的顶部就是,不管这个函数发生什么

03:17 is assumed by torch to never require any gradients. So it will not do any of the bookkeeping
被火炬假定为永远不需要任何梯度。所以它不会做任何簿记

03:24 that it does to keep track of all the gradients in anticipation of an eventual backward pass.
以便追踪所有梯度 以预知最终会有一个后向通行证

backward pass
反向传播 反向传播
神经网络基础

03:29 It's almost as if all the tensors that get created here have a requires grad of false.
几乎就像所有在这里创造的 变压器都有一个要求的假等级

03:34 And so it just makes everything much more efficient because you're telling torch that I will not call
所以它只会让一切 都变得更有效率 因为你在告诉火炬 我不会打电话

03:38 dot backward on any of this computation and you don't need to maintain the graph under the hood.
任何计算都不要落后 你不需要在引擎盖下保持图示

03:43 So that's what this does. And you can also use a context manager with torch dot no grad.
这就是这个方法。你也可以使用一个背景经理 手插着火炬点,没有研究生。

03:49 And you can look those up. Then here we have the sampling from a model.
你可以查查那些样本 然后我们从模型里找到样品

03:55 Just as before, just a four passive neural net getting the distribution, sampling from it,
和以前一样,只有四个被动神经网 得到分布,从中取样

neural net
神经网络 神经网络
神经网络基础

distribution
分布 分布
数学与微积分

04:00 adjusting the context window and repeating until we get the special end token.
调整上下文窗口和重复,直到我们得到特殊结束符。

04:04 And we see that we are starting to get much nicer looking words sample from the model.
我们看到我们开始得到 更漂亮的外观 字样样本 从模型。

sample
样本 样本
神经网络基础

04:09 That's still not amazing. And they're still not fully named like, but it's much better than
仍然不令人惊叹。他们的名字还没有完全命名, 但是比

04:14 we had with the bi-gram model. So that's our starting point. Now the first thing I would
开始点,我首先想

04:19 like to scrutinize is the initialization. I can tell that our network is very improperly
仔细检查是初始化的首字母。 我可以看出我们的网络非常不正确

04:25 configured at initialization. And there's multiple things wrong with it. But let's
初始化时配置的。 它有多重错误。 但是让我们

04:29 just start with the first one. Look here on the zero iteration, the very first iteration,
先从第一个开始,看看这里零迭代, 第一次迭代,

iteration [ˌɪtəˈreɪʃən]
迭代 模型的权重在训练期间的一次更新。迭代包含计算参数在单批次 数据上的梯度损失。 K k-means 一种热门的聚类 算法，用于对非监督式学习中的样本进行分组。 k-means 算法基 本上会执行以下操作： 以迭代方式确定最佳的 k 中心点（称为形心 ）。 将每个样本分配到最近的形心。与同一个形心距离最近的样本属于同一个组。 k-means
神经网络基础

04:34 we are recording a loss of 27. And this rapidly comes down to roughly one or two or so. So I can
记录到27个损失,很快下降到大约一两个左右。所以我可以

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

04:40 tell that the initialization is almost up because this is way too high. In training of neural
告诉人们启动时间已经快到了 因为这个太高了。在神经训练中

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

04:45 nets, it is almost always the case that you will have a rough idea for what loss to expect
几乎总是这样,你会对损失的预期有粗略想法

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

04:49 at initialization. And that just depends on the loss function and the problem setup.
取决于损失功能和问题设置

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

04:54 In this case, I do not expect 27. I expect a much lower number and we can calculate it together.
在这种情况下,我预计不会27岁,我期望的数字要低得多,我们可以一起计算。

05:00 Basically, at initialization, what we'd like is that there's 27 characters that could come next
基本上,在初始化阶段 我们希望的是 27个字符可以接下来

05:07 for any one training example. At initialization, we have no reason to believe any characters to
初始化时,我们没有理由相信任何字符

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

05:11 be much more likely than others. And so we'd expect that the probability distribution that
可能性比其他人的可能性大得多。所以我们预计

probability
概率 概率
数学与微积分

distribution
分布 分布
数学与微积分

05:15 comes out initially is a uniform distribution assigning about equal probability to all
最初的出出是统一分布, 给所有人分配相同概率

distribution
分布 分布
数学与微积分

probability
概率 概率
数学与微积分

05:21 the 27 characters. So basically what we'd like is the probability for any character
27个字符。所以我们想要的基本上是 任何字符的概率

probability
概率 概率
数学与微积分

05:27 would be roughly one over 27. That is the probability we should record. And then the loss is the
大约超过27, 损失是

probability
概率 概率
数学与微积分

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

05:34 negative log probability. So let's wrap this in a tensor. And then then we can take the log of
负日志概率。 所以让我们把这个包装在一个发压器中, 然后我们可以拿下日志

probability
概率 概率
数学与微积分

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

05:41 it. And then the negative log probability is the loss we would expect, which is 3.29,
负日志概率是 我们预期的损失,也就是3.29

probability
概率 概率
数学与微积分

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

05:47 much, much lower than 27. And so what's happening right now is that at initialization,
远低于27,现在的情况是 初始化时

05:52 the neural net is creating probability distributions that are almost up. Some characters are very
神经网正在产生几近上升的概率分布。有些字符非常

neural net
神经网络 神经网络
神经网络基础

probability
概率 概率
数学与微积分

05:57 confident and some characters are very not confident. And then basically what's happening is
和一些人物非常不自信。然后基本上发生了什么是

06:01 that the network is very confidently wrong. And that makes that's what makes it record
网络非常自信地错误。这让它记录下来

06:09 very high loss. So here's a smaller four dimensional example of the issue. Let's say
损失非常高。下面是这个问题的四维小例子,我们假设

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

06:13 we only have four characters. And then we have logits that come out of the neural net and they
我们只有四个字符。然后我们有登录 出神经网,他们

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

neural net
神经网络 神经网络
神经网络基础

06:18 are very, very close to zero. Then when we take the softmax of all zeros, we get probabilities
接近零,然后当我们拿下所有零的软峰值时, 我们就会得到概率

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

06:24 that are a diffuse distribution. So sums to one and is exactly uniform. And then in this case,
它们是分散的。一个加起来,一个整齐,一模一样。然后,在这种情况下,

distribution
分布 分布
数学与微积分

06:32 if the label is say two, it doesn't actually matter if this if the label is two or three
如果标签是2, 则如果标签是2或3, 这实际上并不重要

label ['leibl]
标签 在监督式学习中，标签指样本 的 " 答案 " 或 " 结果 " 部分。有标签数据集中的每个样本 都包含一个或多个特征以及一个标签。例如，在房屋数据集中，特征可能包括卧室数、卫生 间数以及房龄，而标签则可能是房价。在垃圾邮件检测数据集中，特征可能包括主题行、发件人以及电子邮件本身，而标签则可能是 " 垃圾邮件 " 或 " 非垃圾邮件 "
神经网络基础

06:37 or one or zero, because it's a uniform distribution, we're recording the exact same
或一零,因为它是统一的分布, 我们记录完全相同的

distribution
分布 分布
数学与微积分

06:40 loss. In this case, 1.38. So this is the loss we would expect for a four dimensional example.
在本案中,损失为1.38。这就是我们所期望的四维例子的损失。

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

06:46 And I can see of course that as we start to manipulate these logits, we're going to be
我当然可以看出来 当我们开始操纵这些登入时 我们将会

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

06:50 changing the loss here. So it could be that we lock out and by chance, this could be a very
换掉损失,所以我们可能被锁起来 偶然地,这很可能是一个非常

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

06:56 high number like, you know, five or something like that. Then in that case, we'll record a very
高数字,比如,你知道,五或类似。

07:00 low loss because we're signing the correct probability at initialization by chance
低损失是因为我们正偶然在初始化时 签下正确概率

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

probability
概率 概率
数学与微积分

07:04 to the correct label. Much more likely it is that some other dimension will have a high
更可能的是,某些其他维度的值会高一些

label ['leibl]
标签 在监督式学习中，标签指样本 的 " 答案 " 或 " 结果 " 部分。有标签数据集中的每个样本 都包含一个或多个特征以及一个标签。例如，在房屋数据集中，特征可能包括卧室数、卫生 间数以及房龄，而标签则可能是房价。在垃圾邮件检测数据集中，特征可能包括主题行、发件人以及电子邮件本身，而标签则可能是 " 垃圾邮件 " 或 " 非垃圾邮件 "
神经网络基础

07:12 logit. And then what will happen is we start to record much higher loss. And what can come,
登录。然后会发生什么,我们开始记录更多的损失。而接下来会发生什么,

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

07:17 what can happen is basically the logits come out like something like this, you know, and they
基本上,可以发生的事情是 日志出来像这样的东西, 你知道,他们

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

07:22 take on extreme values and we record really high loss. For example, if we have torched up
以极端值取极值,我们记录了非常高的损失。例如,如果我们烧了

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

07:29 random of four, so these are uniform, so these are normally distributed numbers for them.
随机数为 4, 所以这些是统一的, 所以通常会为他们分配数字。

07:40 And here we can also print the logits probabilities that come out of it and loss. And so because
在这里,我们也可以打印 日志的概率 由此而出和损失。所以因为

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

07:47 these logits are near zero, for the most part, the loss that comes out is is okay.
这些登机记录接近零, 大部分情况下, 损失是好的。

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

07:53 But suppose this is like times 10 now, you see how because these are more extreme values,
但假设这是现在的乘以10, 你看这些是更极端的价值观

08:01 it's very unlikely that you're going to be guessing the correct bucket. And then you're
猜对了水桶,你不可能猜对了

08:06 confidently wrong and recording very high loss. If your logits are coming out even more extreme,
自信地错误,记录了非常高的损失。如果你们的登录更加极端,

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

08:12 you might get extremely insane losses like infinity even at initialization.
即使在初始化时,也会遭受无限的疯狂损失。

08:20 So basically, this is not good. And we want the logits to be roughly zero
所以,基本上说,这并不好。 我们希望日志大约为零

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

08:25 when the network is initialized. In fact, the logits can don't have to be just zero,
当网络初始化时。事实上, 登录不能是零,

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

08:29 they just have to be equal. So for example, if all the logits are one,
他们必须平等。例如,如果所有记录都是同一种记录的话,

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

08:34 then because of the normalization inside the softmax, this will actually come out okay.
并且因为软体内正常化, 这实际上会出来。

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

08:38 But by symmetry, we don't want it to be any arbitrary positive or negative number,
但是,通过对称, 我们不希望它是 任意的正或负数,

08:41 we just want it to be all zeros and record the loss that we expect at initialization.
我们只想让它成为零 记录我们初始化时预期的损失

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

08:46 So let's not concretely see where things go wrong in our example.
所以,我们不要具体地看到我们的事例哪里出了问题。

08:49 Here we have the initialization. Let me re-initialize the neural net.
开始启动,让我重新启动神经网

neural net
神经网络 神经网络
神经网络基础

08:53 And here let me break after the very first iteration. So we only see the initial loss,
第一次迭代之后,让我打破这里。所以我们只看到最初的损失,

iteration [ˌitəˈreiʃən]
迭代 模型的权重在训练期间的一次更新。迭代包含计算参数在单批次 数据上的梯度损失。 K k-means 一种热门的聚类 算法，用于对非监督式学习中的样本进行分组。 k-means 算法基 本上会执行以下操作： 以迭代方式确定最佳的 k 中心点（称为形心 ）。 将每个样本分配到最近的形心。与同一个形心距离最近的样本属于同一个组。 k-means
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

08:57 which is 27. So that's way too high. And intuitively, now we can expect the variables
27,所以这太高了。直觉上,现在我们可以预期变量

09:03 involved. And we see that the logits here, if we just print some of these,
并且我们可以看到这里的登录记录 如果我们只是打印一些这些

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

09:09 if we just print the first row, we see that the logits take on quite extreme values.
如果我们只打印第一行,我们就会看到 登录具有非常极端的值。

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

09:13 And that's what's creating the fake confidence and incorrect answers and makes the loss
这就是创造虚假信心和错误答案 并造成损失的原因

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

09:20 get very, very high. So these logits should be much, much closer to zero.
高度非常高 所以这些登录应该离零远得多

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

09:25 So now let's think through how we can achieve logits coming out of this neural net to be more
现在我们来想想我们如何 实现从神经网中产生的登录 来做更多

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

neural net
神经网络 神经网络
神经网络基础

09:30 closer to zero. You see here that the logits are calculated as the hidden states multiplied by W2
接近零。 您可以看到, 日志是用隐藏状态乘以 W2 计算得出的

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

09:36 plus B2. So first of all, currently we're initializing B2 as random values of the right size.
加上B2. 所以首先,我们目前初始化 B2 是随机值的大小。

09:44 But because we want roughly zero, we don't actually want to be adding a bias of random
但是因为我们想要零, 我们其实不想增加一个 随机的偏向

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

09:48 numbers. So in fact, I'm going to add a time to zero here to make sure that B2 is just
数字。事实上,我在这里会增加一个零到零的时间以确保 B2只是

09:54 basically zero at initialization. And second, this is H multiplied by W2. So if we want logits to
初始化时基本为零。 第二,这是H乘以W2。 所以如果我们想要登录到

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

10:01 be very, very small, then we would be multiplying W2 and making that smaller. So for example,
很小,非常小,然后我们就会乘以W2 并缩小。例如,

10:07 if we scale down W2 by 0.1, all the elements, then if I do again just a very first iteration,
如果我们把W2降0.1, 所有元素, 如果我再做一次, 只需要第一次迭代,

iteration [.itə'reiʃən]
迭代 模型的权重在训练期间的一次更新。迭代包含计算参数在单批次 数据上的梯度损失。 K k-means 一种热门的聚类 算法，用于对非监督式学习中的样本进行分组。 k-means 算法基 本上会执行以下操作： 以迭代方式确定最佳的 k 中心点（称为形心 ）。 将每个样本分配到最近的形心。与同一个形心距离最近的样本属于同一个组。 k-means
神经网络基础

10:14 you see that we are getting much closer to what we expect. So roughly what we want is about
你看,我们越来越接近预期。所以大致上,我们想要的就是

10:18 3.29. This is 4.2. I can make this maybe even smaller. 3.32. Okay, so we're getting closer and closer.
329,这是4.2,我可以把这个缩小一点

10:28 Now you're probably wondering, can we just set this to zero? Then we get, of course,
现在你可能想知道,我们能不能把这个设置为零?

10:34 exactly what we're looking for at initialization. And the reason I don't usually do this is because
这正是我们想要的初始化。我通常不这样做的原因是

10:41 I'm very nervous. And I'll show you in a second why you don't want to be setting Ws
我很紧张,我马上告诉你为什么你不想设置WS

10:45 or weights of a neural net exactly to zero. You usually want it to be small numbers instead of
神经网的重量或神经网的加权数 完全为零。通常您希望它数量小而不是

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

neural net
神经网络 神经网络
神经网络基础

10:51 exactly zero. For this output layer in this specific case, I think it would be fine. But I'll show you
完全零。 对于这个特定情况下的输出层, 我想应该没问题。 但我会告诉你

output layer
输出层 神经网络的 " 最后 " 一层，也是包含答案的层。
神经网络基础

10:57 in a second where things go wrong very quickly if you do that. So let's just go with 0.01.
等一秒,如果你这样做的话 事情就会很快出错。所以我们用0.01来计算吧。

11:02 In that case, our loss is close enough, but has some entropy. It's not exactly zero. It's got
那样的话,我们的损失就足够接近了 但有一些的酶。它不是零,它有

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

entropy
熵 熵
数学与微积分

11:08 some low entropy and that's used for symmetry breaking, as we'll see in a second. Logits are
低的酶,用来对称断裂,一秒后我们就会看到。

entropy
熵 熵
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

11:14 now coming out much closer to zero and everything is well and good. So if I just erase these and I'd
现在,我走出来 距离零点更近了 一切都很好好。所以如果我把这些擦掉,我会

11:21 now take away the break statement, we can run the optimization with this new initialization.
现在取消休息声明, 我们可以运行优化 与这一新初始化。

11:28 And let's just see what losses we record. Okay, so I'll let it run. And you see that we started
让我们看看我们记录了哪些损失。好,所以我让它运行。你看,我们开始了

11:34 off good. And then we came down a bit. The plot of the loss now doesn't have this hockey
之后我们跌下来一点,现在损失的情节没有冰球

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

11:41 shape appearance. Because basically what's happening in the hockey stick, the very first few iterations
外观形状。 因为基本上曲棍球棒中发生了什么, 最初的几处迭代

11:47 of the loss, what's happening during the optimization is the optimization is just
损失的损失, 最佳化期间发生的事情是 优化只是

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

11:51 squashing down the logits and then it's rearranging the logits. So basically we took away this easy
压碎日志,然后重新排列日志。所以基本上我们把这个容易的拿走了

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

11:56 part of the loss function where just the weights were just being shrunk down. And so therefore
部分损失功能, 就是仅仅重量的重量被缩小。所以

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

12:02 we don't get these easy gains in the beginning and we're just getting some of the hard gains
我们一开始并没有得到这些容易的收获, 而我们只是得到一些艰难的收获

12:07 of training the actual neural net. And so there's no hockey stick appearance. So good things are
训练真正的神经网,所以没有曲棍球棒外表。

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

neural net
神经网络 神经网络
神经网络基础

12:12 happening in that both number one loss initialization is what we expect. And the loss doesn't look like
两种损失的首选 和我们所期望的一样

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

12:19 a hockey stick. And this is true for any neural net you might train and something to look out for.
任何你可能训练的神经网和需要注意的东西都是如此

neural net
神经网络 神经网络
神经网络基础

12:25 And second, the loss that came out is actually quite a bit improved. Unfortunately,
第二,损失的减少实际上有相当改善。不幸的是,

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

12:30 I erased what we had here before. I believe this was 2.12 and this was 2.16. So we get a slightly
我删除了我们以前在这里的物品, 我相信这是2.12和2.16。

12:38 improved result. And the reason for that is because we're spending more cycles, more time,
之所以这样是因为我们花费了更多的周期,更多的时间

12:44 optimizing the neural net actually, instead of just spending the first several thousand iterations
而不是仅仅花掉前几千次的迭代

neural net
神经网络 神经网络
神经网络基础

12:49 probably just squashing down the weights because they are so way too high in the beginning
可能只是压压重量 因为它们太高 在开始的时候

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

12:55 and the initialization. So something to look out for and that's number one. Now let's look at the
和初始化。所以有些东西需要注意,这是第一号。现在让我们来看看

13:00 second problem. Let me re initialize our neural net and let me reintroduce the break statement.
第二个问题。让我重新开始我们的神经网, 让我重新做一次休息发言。

neural net
神经网络 神经网络
神经网络基础

13:06 So we have a reasonable initial loss. So even though everything is looking good on a level
因此我们有一个合理的初始损失。所以即使一切看起来都不错

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

13:10 of the loss and we get something that we expect, there's still a deeper problem
损失的损失和我们得到的东西 我们期望,还有更深的问题

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

13:14 lurking inside this neural net and its initialization. So the logits are now okay. The
潜伏在这个神经网中,并开始启动。所以日志现在可以了。

neural net
神经网络 神经网络
神经网络基础

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

13:19 problem now is with the values of H, the activations of the hidden states. Now if we just
现在的问题是H的值, 隐藏状态的激活。 如果我们只是

链式法则 · 反向传播 13:25

13:25 visualize this vector, sorry, this tensor H, it's kind of hard to see but the problem here roughly
想象一下这个矢量,对不起,这个抗H, 它有点难以看到 但这里的问题大致上

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

13:31 speaking is you see how many of the elements are one or negative one. Now recall that torch.10h,
正在说话的您是否看到其中有多少元素是其中的一个或负的。 现在请记住,火炬10h,

13:37 the 10h function is a squashing function. It takes arbitrary numbers and it squashes them
10h 函数是一个压碎函数。 它需要任意数字, 并粉碎它们

13:42 into a range of negative one and one and it does so smoothly. So let's look at the histogram
进入一个负一和一个的范围,它非常顺利。让我们看看直方图

13:47 of H to get a better idea of the distribution of the values inside this tensor. We can do this first.
H 来更好地了解这个阵列内值分布的分布情况。 我们可以先做这件事。

distribution
分布 分布
数学与微积分

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

13:54 Well, we can see that H is 32 examples and 200 activations in each example. We can view it as
我们可以看到H是32个例子和200个激活。

14:01 negative one to stretch it out into one large vector and we can then call tool list to convert
负1 将它拉伸成一个大的矢量,然后我们就可以调用工具列表转换

14:09 this into one large Python list of floats. And then we can pass this into plt.hist for
将它变成一个巨大的 Python 浮点数列表。 然后我们就可以把这个传递到 plt.

14:16 histogram and we say we want 50 bins and a semicolon to suppress a bunch of output we don't want.
直方图,我们说我们要50个垃圾箱和一个分号 来压制我们不希望的一堆输出

14:24 So we see this histogram and we see that most of the values by far take on value of negative one
我们看到这个直方图,我们看到大多数值 都取负值

14:29 and one. So this 10h is very, very active and we can also look at basically why that is,
和一。所以这10小时非常,非常活跃 我们也可以研究一下为什么

14:37 we can look at the preactivations that feed into the 10h and we can see that the
我们可以看一看10小时的预作用 可以看到

14:43 distribution of the preactivations are is very, very broad. These take numbers between negative
预引爆分布非常、非常广泛,从负数到负数

distribution
分布 分布
数学与微积分

14:48 15 and 15 and that's why in a torcha 10h everything is being squashed and capped to be in the range
15和15,所以在10小时的火炬中 所有的东西都被压扁 并被封盖到射程

14:54 of negative one and one and lots of numbers here take on very extreme values. Now if you are new
和很多数字中,这里的负数和负数 都具有非常极端的值。如果您是新

14:59 to neural networks, you might not actually see this as an issue, but if you're well-burst in
神经网络,你可能不会实际 将这看成是一个问题 但如果你心情好

15:04 the dark arts back propagation and then have an intuitive sense of how these gradients flow
黑暗艺术回回传播 然后有直觉感知 这些梯度是如何流动的

back propagation
反向传播 反向传播
神经网络基础

15:08 through in your own net, you are looking at your distribution of 10h activations here and you are
透过自己的网,你正在看着 你分布的10小时启动器在这里

distribution
分布 分布
数学与微积分

15:13 sweating. So let me show you why. We have to keep in mind that during back propagation, just like we
汗流出汗,所以让我告诉你为什么。我们必须记住,在后面的传播过程中,就像我们一样

back propagation
反向传播 反向传播
神经网络基础

15:18 saw in micrograd, we are doing backward pass starting at the loss and flowing through the
微格中我们从损失开始 后退,然后流过

backward pass
反向传播 反向传播
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

15:22 network backwards. In particular, we're going to back propagate through this torcha 10h
网络向后向。 特别是, 我们打算通过这个 10h 的火炬回传

15:28 and this layer here is made up of 200 neurons for each one of these examples
并且这个层由200个神经元组成 每个例子都是这样

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

15:33 and it implements an element-wise 10h. So let's look at what happens in 10h in the
并执行一个元素的10小时。让我们来看看在10小时后会发生什么

15:38 backward pass. We can actually go back to our previous micrograd code in the very first lecture
后向过往的通道。我们可以回到我们第一次讲座上之前的微观学分代码

backward pass
反向传播 反向传播
神经网络基础

15:44 and see how we implement a 10h. We saw that the input here was x and then we calculate t which is
并查看我们如何执行一个10小时。我们看到输入是x,然后我们计算出

15:50 the 10h of x. So that's t and t is between negative one and one, it's the output of the 10h
x 10 小时。 所以,这是 T 和 t 之间的负 1 和 1, 这是 10 分钟的输出

15:56 and then in the backward pass how do we back propagate through a 10h? We take out that grad
然后在后退的球道上 我们如何回到10小时之间传播?

backward pass
反向传播 反向传播
神经网络基础

16:02 and then we multiply it this is the chain rule with the local gradient which took the form
然后我们乘以它,这是带本地梯度的链条规则 其形式是

chain rule
链式法则 链式法则
反向传播与梯度

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

16:06 of 1 minus t squared. So what happens if the outputs of your 10h are very close to negative one or one?
1 减 t 平方。 那么,如果您的10小时输出非常接近负一个或一个结果,会发生什么?

16:13 If you plug in t equals one here, you're going to get a zero multiplying out that grad. No matter
如果你插入 t 等于 1, 你会得到一个零乘数 从那个研究生。

16:19 what out that grad is, we are killing the gradient and we're stopping effectively the
毕业后我们正在杀死梯度 我们有效地停止了

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

16:24 back propagation through this 10h unit. Similarly, when t is negative one, this will again become
与此类似,当 t为负时,它会再次成为

back propagation
反向传播 反向传播
神经网络基础

16:29 zero and out that grad just stops and intuitively this makes sense because this is a 10h
毕业后零零,刚停止,直觉地说 这有道理,因为这是一个10小时

16:35 neuron and what's happening is if its output is very close to one then we are in the tail of this 10h
神经元和正在发生的情况是 如果它的输出非常接近一个, 那么我们就会在10小时的尾端

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

16:43 and so changing basically the input is not going to impact the output of the 10h too much
基本上,输入不会对10小时的输出产生太大的影响

16:52 because it's in a flat region of the 10h and so therefore there's no impact on the loss
因为它位于10小时平原地区 所以对损失没有影响

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

16:58 and so indeed the weights and the biases along with this 10h neuron do not impact the loss
所以,确实的重量和偏差 加上这10小时神经元

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

17:05 because the output of this 10h unit is in the flat region of the 10h and there's no influence.
因为这10小时单位的输出 在10小时平坦的地区,没有影响。

17:09 We can be changing them however we want and the loss is not impacted. That's another way to justify
换他们,不管我们想怎样,损失不会受到影响。

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

17:15 that indeed the gradient would be basically zero. It vanishes. Indeed when t equals zero
梯度必定是零,而零位消失。当它等于零时,

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

17:24 we get one times out that grad so when the 10h takes on exactly value of zero
我们从那个研究生那里得到一次机会 所以当10小时的精确值为零时

17:31 then out that grad is just passed through. So basically what this is doing is if t is equal to zero
然后,毕业后,刚通过。所以基本上,这是如果t等于零

17:38 then the 10h unit is sort of inactive and gradient just passes through but the more you are in the
10h 单位是非活动单位, 梯度只是通过, 但越多您在

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

17:45 flat tails the more the gradient is squashed. So in fact you'll see that the gradient flowing
斜度被压扁越多, 平面就会平平板。 所以事实上你会看到渐梯流

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

17:52 through 10h can only ever decrease and the amount that it decreases is proportional
至10小时只能减少,其减少的金额是成比例的

17:58 through a square here depending on how far you are in the flat tails of this 10h
取决于你在这10小时平面尾巴里 有多远

18:05 and so that's kind of what's happening here and through this the concern here is that if all of
所以,这就是 发生在这里和 通过这个事情

18:10 these outputs h are in the flat regions of negative one and one then the gradients that are flowing
这些输出h位于负数平坦区域,一个平坦区域为负数区域,然后是流动的梯度区域。

18:16 through the network will just get destroyed at this layer. Now there is some redeeming
网络的这个层会被摧毁

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

18:22 quality here and that we can actually get a sense of the problem here as follows.
这里的质量,我们实际上可以 得到一个问题感知 在这里如下。

18:27 I wrote some code here and basically what we want to do here is we want to take a look at
我在这里写了一些代码 基本上我们想在这里做的是 我们想看看

18:32 h take the absolute value and see how often it is in the in the flat region so say greater than 0.99
h 表示绝对值,看看在平坦地区多频繁,所以说大于0.99。

18:42 and what you get is the following and this is a Boolean tensor so in the Boolean tensor you get
和得到的是以下的, 这是布尔式的 布拉式的

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

18:48 a white if this is true and a black if this is false and so basically what we have here is
白色的,如果这是真的,黑色的,如果是假的,所以基本上我们在这里是什么

S 型函数 · 线性修正单元 18:53

18:53 the 32 examples and the 200 hidden neurons and we see that a lot of this is white and what that's
32个例子和200个隐藏的神经元 我们发现很多这些是白色的

19:00 telling us is that all these 10h neurons were very very active and they're in a flat tail
告诉我们,所有10个小时的神经元都非常活跃 它们处于平坦的尾巴

19:09 and so in all these cases the back the backward gradient would get destroyed.
所以在所有这些情况下 后退梯度就会被摧毁

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

19:16 Now we would be in a lot of trouble if for any one of these 200 neurons if it was the case
如果真的发生的话 这两百个神经元中的任何一颗 都有可能陷入麻烦

19:23 that the entire column is white because in that case we have what's called a dead neuron
整个列的白色是白色 因为在这种情况下 我们有一个叫做死神经元

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

19:28 and this could be a 10h neuron where the initialization of the weights in the biases
这可能是10小时的神经元, 开始偏差中的重量

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

19:32 could be such that no single example ever activates this 10h in the sort of active part
可能没有任何一个例子能激活这10小时的主动部分

19:38 of the 10h. If all the examples land in the tail then this neuron will never learn it is a dead
如果所有的例子都落在尾尾部的地上 那么这个神经元永远也学不到它已经死了

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

19:45 neuron and so just scrutinizing this and looking for columns of completely white we see that
寻找完全白色的柱子 我们可以看到

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

19:52 this is not the case so I don't see a single neuron that is all of you know white and so
情况不是这样 所以我看不到一个神经元 你们都知道白的

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

19:59 therefore it is the case that for every one of these 10h neurons we do have some examples that
因此,对于这10个小时神经元中的每一个,我们确实有一些例子可以说明

20:05 activate them in the active part of the 10h and so some gradients will flow through and this neuron
激活在10小时的活性部分, 所以有些梯度会流过, 这个神经元

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

20:11 will learn and the neuron will change and it will move and it will do something but you can
神经元会改变,神经元会变 它会移动,它会做一些事 但你可以

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

20:16 sometimes get yourself in cases where you have dead neurons and the way this manifests is that
有时在有死神经元的情况下 出现这样的情况

20:22 for a 10h neuron this would be when no matter what inputs you plug in from your data set this
10h 神经元的 10 小时, 这将是无论您从数据集中插入什么输入都无法

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

20:27 10h neuron always fires completely one or completely negative one and then it will just not learn
10个h神经元总是完全或完全负的, 完全燃烧一个或者完全负的 然后它就会不学

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

20:33 because all the gradients will be just zeroed out. This is true not just for 10h but for a lot
因为所有梯度都会被全部除以零。 这不仅对10小时,而且对很多时间都是如此

20:38 of other non-linearities that people use in neural networks so we certainly use 10h a lot
神经网络中人们使用的其他非线性信息 所以我们当然用10个小时

20:43 but sigmoid will have the exact same issue because it is a squashing neuron and so the
因为它是压碎的神经元 所以

sigmoid
S 型函数 S 型函数
数学与微积分

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

20:47 same will be true for sigmoid but basically the same will actually apply to sigmoid. The same
类类体将同样适用,但基本上相同将实际适用于类类体。

sigmoid
S 型函数 S 型函数
数学与微积分

20:57 will also apply to a relu so relu has a completely flat region here below zero so if you have a
也同样适用于在零以下完全平平的relu so relu区域,如果

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

21:03 relu neuron then it is a pass-through if it is positive and if the pre-activation is negative
瑞鲁神经元,如果是正的,如果预激活为负的,则是一种自由通过

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

21:11 it will just shut it off since the region here is completely flat then during back propagation
因为它会关闭它 因为这里的区域完全平整 然后在后面的传播中

back propagation
反向传播 反向传播
神经网络基础

21:17 this would be exactly zeroing out the gradient like all of the gradient would be set exactly to
将完全除去梯度, 就像所有梯度都设置在

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

21:22 zero instead of just like a very very small number depending on how positive or negative t is and so
而不是像一个非常小的数量一样,取决于正或负是多少还是负 t

21:28 you can get for example a dead relu neuron and a dead relu neuron would basically look like
比如说,你可以得到一个死性Relu神经元 和一个死性 Relu神经元

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

21:35 basically what it is is if a neuron with a relu non-linearity never activates so for any

基本上它是什么 如果一个神经元 与无线性共聚物的神经元 从来没有像任何

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

21:42 examples that you plug in in the dataset it never turns on it's always in this flat region

插插在数据集中的示例,它从未打开过它, 它总是在这个平坦的区域

21:47 then this relu neuron is a dead neuron its weights and bias will never learn they will never get a

并且偏见永远不会知道 他们永远得不到

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

21:52 gradient because the neuron never activated and this can sometimes happen at initialization

梯度,因为神经元从未激活,有时在初始化时会发生

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

21:57 because the weights in a bias is just make it so that by chance some neurons are just forever
因为偏偏的重量 仅仅是使它, 偶然一些神经元 只是永远

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

22:01 dead but it can also happen during optimization if you have like a too high of a learning
死后,它也可以在优化期间发生 如果你有像一个学习太高

22:06 rate for example sometimes you have these neurons that gets too much of a gradient
例如,比率有时有这样的神经元 得到的梯度太大

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

22:10 and they get knocked out of data manifold and what happens is that from then on no example ever
数据流被挤出,结果从此以后

22:16 activates this neuron so this neuron remains dead forever so it's kind of like a permanent brain
激活这个神经元 这样这个神经元就永远死去 所以它有点像一个永久性的大脑

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

22:20 damage in a in a mind of a network and so sometimes what can happen is if your learning
网络的脑中受损 并且有时,如果学习

22:25 rate is very high for example and you have a neural net with relu neurons you train the
例如,你有一个神经网, 配有可塑性神经元, 你训练

neural net
神经网络 神经网络
神经网络基础

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

22:30 neural net and you get some last loss but then actually what you do is you go through the
最终失去一些最后的神经网络 但实际上你所做的就是通过

neural net
神经网络 神经网络
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

22:34 entire training set and you forward your examples and you can find neurons that never activate
整个训练设置,然后您转发示例,然后找到从未激活的神经元

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

22:41 they are dead neurons in your network and so those neurons will will never turn on
它们是你们网络中死去的神经元 所以这些神经元永远不会打开

22:46 and usually what happens is that during training these relu neurons are changing moving etc
通常情况下,在训练过程中 这些雷鲁神经元正在变化等等

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

22:50 and then because of a high gradient somewhere by chance they get knocked off
之后,因为高梯度 某个地方,他们偶然被打倒

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

22:54 and then nothing ever activates them and from then on they are just dead
没有任何东西能激活它们,而从那以后,它们就只是死的;

22:58 so that's kind of like a permanent brain damage that can happen to some of these neurons
这是一种永久性的脑损伤 可能发生在一些神经元身上

23:03 these other nonlinearities like leaky relu will not suffer from this issue as much
这些问题不会受这个问题的影响,

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

23:07 because you can see that it doesn't have flat tails you'll almost always get gradients
因为你可以看到它没有平面尾巴 你几乎总是会得到梯度

23:12 and ilu is also fairly frequently used it also might suffer from this issue because it has
并且伊卢也相当经常使用,它也可能由于它曾经受到过这种影响而受到影响。

23:18 flat parts so that's just something to be aware of and something to be concerned about
平板部分,所以这只是一些需要注意的事情和关注的事

23:24 and in this case we have way too many activations h that take on extreme values
在这种情况下,我们有太多的激活h 具有极端价值

23:30 and because there's no column of white I think we will be okay and indeed the network optimizes
因为没有白色的一列 我认为我们将会没事,网络确实优化

23:35 and gives us a pretty decent loss but it's just not optimal and this is not something you want
给我们带来相当体面的损失 但这不是最佳的 这不是你想要的

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

23:40 especially during initialization and so basically what's happening is that this
尤其是在初始化阶段,所以基本上发生了什么是

23:45 h pre activation that's flowing to 10 h it's it's too extreme it's too large it's creating
h 预激活,流到10 h 它太极端了 它创造的太大

activation
激活值 激活值
神经网络基础

23:51 very it's creating a distribution that is too saturated in both sides of the 10 h
因为它在10小时的两侧 制造出一种过于饱和的分布

distribution
分布 分布
数学与微积分

23:57 and it's not something you want because it means that there's less training
这不是你想要的东西 因为你想要的东西 因为它意味着,有少了训练

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

24:01 for these neurons because they update less frequently so how do we fix this well h
因为这些神经元的更新频率较低,所以我们如何修补这个井 h

24:07 pre activation is mcat which comes from c so these are uniform Gaussian but then multiply by w1
启动前是来自c的 mcat , 所以这些是制服高山, 但乘以 w1

activation
激活值 激活值
神经网络基础

24:16 plus b1 and h preact is too far off from zero and that's causing the issue so we want this
加上 b1 和 hhpact 与零太远, 离零太远了, 这正在引起问题 所以我们想要这个

24:22 pre activation to be closer to zero very similar to what we had with logits so here we won't actually
与我们用登录记录 接近于零,所以在这里 我们不会真的

activation
激活值 激活值
神经网络基础

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

24:29 something very very similar now it's okay to set the biases to a very small number we can either
现在,我们也可以把偏向 定在极少数人身上

24:35 multiply by zero zero one to get like a little bit of entropy I sometimes like to do that just so
乘以零零 1 以获得像一点点的 entropy 我有时喜欢这样做 只是这样

entropy
熵 熵
数学与微积分

24:41 that there's like a little bit of variation and diversity in the original initialization of these
最初的初始化过程 有一些差异和多样性

24:48 10 h neurons and I find in practice that that can help optimization a little bit and then the
10个神经元和我发现在实践中, 能够帮助优化一点点 然后

24:54 weights we can also just look squash so let's multiply everything by point one let's rerun the first batch
重量也只能看壁球,所以让我们把每件事情乘以一点 一点再重新运行第一批

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ， 则相应的特征对模型来说没有任何贡献。
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

25:01 and now let's look at this and well first let's look here
现在让我们来看看这个,好 让我们先看看这里

25:06 you see now because we multiply it double by point one we have a much better histogram
因为我们把它乘以一乘倍, 我们的直方图更好

25:11 and that's because the pre activations are now between negative 1.5 and 1.5
这是因为预启动在负1.5到1.5之间

25:14 and this we expect much much less white okay there's no white so basically that's because there are no
并且我们期望的白色要少得多, 白的要少得多。 好吧,没有白色 所以基本上,因为没有

25:23 neurons that saturated above 0.99 in either direction so it's actually a pretty decent place to be
顺着两个方向的0.99以上神经元 都饱和在0.99以上 所以它实际上是一个不错的地方

25:31 maybe we can go up a little bit sorry am I changing w1 here so maybe we can go to 0.2
也许我们可以上去一下 对不起 我在这里换W1 这样我们就可以去0.2

25:40 okay so maybe something like this is a nice distribution so maybe this is what our initialization
所以,这样的东西 像这样的发行是不错的 所以也许这就是我们初始化

distribution
分布 分布
数学与微积分

25:48 should be so let me now erase these and let me starting with initialization let me run the
现在让我把这些都抹掉 让我从初始化开始 让我来运行

25:57 full optimization without the break and let's see what we get okay so the optimization finished
完全优化,不休息 让我们看看我们有什么好结果 这样就完成了优化

26:04 and I rerun the loss and this is the result that we get and then just as a reminder I put
我重新恢复了损失,这就是我们得到的结果, 然后就像我提醒的一样

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

26:09 down all the losses that we saw previously in this lecture so we see that we actually do get an
损失了我们以前在本次讲座上看到的所有损失 因此我们看到,我们确实得到了

26:14 improvement here and just as a reminder we started off with a validation loss of 2.17 when we started
并且作为提醒 我们一开始的验证损失是2.17 当我们开始的时候

validation
验证 验证
训练与优化

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

26:19 by fixing the softmax being confidently wrong we came down to 2.13 and by fixing the 10h
修补软峰,安心地错误 我们跌到2.13,修整10

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

26:25 layer being way too saturated we came down to 2.10 and the reason this is happening of course
我们下到2.10,当然原因

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

26:30 is because our initialization is better and so we're spending more time doing productive training
因为我们的初始化更好 所以我们花更多的时间做生产性培训

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

26:34 instead of not very productive training because our gradients are set to zero and we have to learn
而不是因为我们梯度定在零,我们必须学习而没有进行非常生产性的培训

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

26:41 very simple things like the overconfidence of the softmax in the beginning and we're spending cycles
简单简单的事情 比如一开始软体的自信过强 而我们是支出周期

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

26:46 just like squashing down the weight matrix so this is illustrating basically initialization
就像压压减重量矩阵,所以这说明 基本上初始化

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

matrix
矩阵 矩阵
数学与微积分

26:53 and its impacts on performance just by being aware of the internals of these neural nets and
了解这些神经网的内部,对工作绩效的影响

26:58 their activations and their gradients now we're working with a very small network this is just
他们的活性动力和梯度 现在我们正在与一个很小的网络合作

27:03 one layer multi layer perception so because the network is so shallow the optimization problem is
一层多层的感知,所以因为网络太浅,优化问题就是

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

27:08 actually quite easy and very forgiving so even though our initialization was terrible the network
网络的初始化很可怕

27:13 still learned eventually it just got a bit worse result this is not the case in general though
最终还是学到了,结果却更糟 但一般情况并非如此

27:19 once we actually start working with much deeper networks that have say 50 layers things can get
一旦我们开始与更深得多的网络合作 说有50层的网络

27:25 much more complicated and these problems stack up and so you can actually get into a
这些问题堆叠起来,所以你可以进入一个

27:32 place where the network is basically not training at all if your initialization is bad enough and
网络基本上没有被训练的地方 如果你的初始化已经够差的话

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

27:37 the deeper your network is and the more complex it is the less forgiving it is to some of these errors
越深你的网络越复杂 其中一些错误就越不宽宏大量

27:42 and so something to be definitely be aware of and something to scrutinize something to plot
并且有东西可以肯定地知道 和仔细研究的东西来密谋

线性修正单元 · 隐藏层 27:49

27:49 and something to be careful with and yeah okay so that's great that that worked for us
和一些要小心的东西 与,是的,好 所以这是伟大的 那对我们有用

27:55 but what we have here now is all these magic numbers like 0.2 like where do I come up with
但现在我们这里的 却是所有这些神奇的数字 像0.2,比如我从哪里找到

27:59 this and how am I supposed to set these if I have a large neural network lots and lots of layers
如果我有庞大的神经网络 有很多层和很多层,我该如何设置这些

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

28:05 and so obviously no one does this by hand there's actually some relatively principled ways of setting
所以显然没有人亲手这样做 实际上有一些相对有原则的方法 来设定

28:09 these scales that I would like to introduce to you now so let me paste some code here that I
这些尺度,我现在要向你们介绍一下。 让我在这里粘贴一些代码,

28:15 prepared just to motivate the discussion of this so what I'm doing here is we have some
准备激发讨论的动力 所以我在这里要做的就是 我们有一些

28:21 random input here x that is drawn from a Gaussian and there's 1000 examples that are 10
这里的随机输入 x x , 从高西亚语中抽取, 1,000个示例为 10

28:27 dimensional and then we have a weight in layer here that is also initialized using Gaussian just
维度,然后我们有一个在层中的重量 这里的一层,这个也使用高森

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ， 则相应的特征对模型来说没有任何贡献。
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

28:33 like we did here and we hit these neurons in the hidden layer look at 10 inputs and there are 200
就像我们在这里所做的,我们打中了这些神经元 在隐藏层的隐形层 看着10个输入,有200个

hidden layer
隐藏层 神经网络 中的合成层，介于输入层 （即特征）和输出层 （即预测）之间。神经网络包含一个或多个隐藏层。
神经网络基础

28:39 neurons in this hidden layer and then we have here just like here in this case the multiplication
隐藏在这种隐蔽层中的神经元 然后我们在这里 和这里一样,在这个案例中

hidden layer
隐藏层 神经网络 中的合成层，介于输入层 （即特征）和输出层 （即预测）之间。神经网络包含一个或多个隐藏层。
神经网络基础

28:46 x multiplied by w to get the pre-activations of these neurons and basically the analysis here
xx乘以 w 来获得这些神经元的预激活 以及基本的分析

28:52 looks at okay suppose these are uniform Gaussian and these weights are uniform Gaussian if I
假设这些是制服高森式的 重量也是制服高森型的 如果我

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

28:57 do x times w and we forget for now the bias and the non-linearity then what is the mean and the
做xxxx次,我们忘记了偏见和非线性 然后什么是用

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

29:04 standard deviation of these Gaussians so in the beginning here the input is just a normal Gaussian
这些高斯人的标准差 所以在开始的时候输入只是普通的高斯人

29:10 distribution mean 0 and the standard deviation is 1 and the standard deviation again is just
标准差为 1,而标准差又只是

distribution
分布 分布
数学与微积分

29:14 the measure of a spread of the Gaussian but then once we multiply here and we look at the
测量高山的扩张范围 但一旦我们在此成倍增长 我们就会看到

29:21 histogram of y we see that the mean of course stays the same it's about 0 because this is a
y的直方图 我们发现,这个平均值当然保持不变 大约是0

29:27 symmetric operation but we see here that the standard deviation has expanded to 3 so the input
对称操作, 但我们在这里看到标准偏差已经扩大至 3, 所以输入

operation
运算 运算
神经网络基础

29:32 standard deviation was 1 but now we've grown to 3 and so what you're seeing in the histogram is
标准差1分,但现在我们已发展到3分 所以在直方图上你看到的是

29:37 that this Gaussian is expanding and so we're expanding this Gaussian from the input and
高森正在扩张 所以我们从投入和

29:45 we don't want that we want most of the neural nets to have relatively similar activations so
我们不希望我们想要 大部分神经网 具有相对相似的激活装置

29:50 unit Gaussian roughly throughout the neural net as the question is how do we scale these w's to
问题是我们如何扩大这些

neural net
神经网络 神经网络
神经网络基础

29:59 preserve this distribution to remain a Gaussian and so intuitively if I multiply here these
保存此分布以保持高西亚的特性, 如果我在此乘这些

distribution
分布 分布
数学与微积分

30:07 elements of w by a large number like say by 5 then this Gaussian grows and grows
以一大批量的元素,比如说5到5, 然后这个高山生长

30:14 and standard deviation so now we're at 15 so basically these numbers here in the output y
和标准偏差 所以现在我们只有15,所以基本上这些数字 在输出y中

30:20 take on more and more extreme values but if we scale it down like say 0.2 then conversely
越来越极端的值 但是如果我们像0.2一样缩小它,那么反过来

30:26 this Gaussian is getting smaller and smaller and it's shrinking and you can see that the standard
高山越来越小,越来越小 它正在缩小,你可以看到标准

30:32 deviation is 0.6 and so the question is what do I multiply by here to exactly preserve
偏差为0.6,所以问题在于我在这里乘多少来准确保存

30:38 the standard deviation to be 1 and it turns out that the correct answer mathematically when
标准差为 1, 并且它证明正确的答案数学上正确

30:42 you work out through the variance of this multiplication here is that you are supposed to
您通过乘法的差异来锻炼出, 这里的乘法是您应该

30:48 divide by the square root of the fan in the fan in is the basically the number of input
扇扇扇中扇扇的平方根除以 基本上输入的数量

30:56 elements here 10 so we are supposed to divide by 10 square root and this is one way to do the
这里的10个元素在这里 10 所以我们应该除以10平方根 这样可以

31:01 square root you raise it to a power of 0.5 that's the same as doing a square root so when you
平方根,您将其提升到0.5的功率, 这与做平方根相同, 当你在

31:07 divide by the square root of 10 then we see that the output Gaussian it has exactly standard
以平方根除以 10 的平方根,然后我们看到输出高西亚有精确的标准

31:15 deviation of 1 now unsurprisingly a number of papers have looked into how but to best initialize neural
现在不令人惊讶的是,一些论文探讨了如何最佳地启动神经系统,但又不能最好地启动神经系统。

31:22 networks and in the case of multivariate perceptrons we can have fairly deep networks that had these
网络,如果是多变式的多光谱,我们可以拥有相当深的网络,这些网络具有

31:27 nonlinearities in between and we want to make sure that the activations are well behaved and
我们之间和我们之间无线性 想要确保激活的运行良好

31:31 they don't expand to infinity or shrink all the way to zero and the question is how do we
问题在于我们如何去

31:36 initialize the weights so that these activations take on reasonable values throughout the network
初始化加权数,使这些激活在整个网络中具有合理的值

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

31:40 now one paper that has studied this in quite a bit detail that is often referenced
现在有一份文件 已经对此进行了相当详细的研究, 经常被引用

31:44 is this paper by Kyming He et al called Delving Deep Interactive Fires now in this case they
本文由Kyming He等人撰写,

31:50 actually study convolutional neural networks and they study especially the ReLU nonlinearity
实际研究进化神经网络, 特别是研究雷卢非线性

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

31:56 and the P ReLU nonlinearity instead of a 10 H nonlinearity but the analysis is very similar
和PRELU的不线性,而不是10H的不线性和不线性。

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

32:01 and basically what happens here is for them the ReLU nonlinearity that they care about quite a bit
基本上这里发生的事情是,他们关心的是 ReLU的无线性

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

线性修正单元 · 前向传播 32:08

32:08 here is a squashing function where all the negative numbers are simply clamped to zero
这里是一个压制函数, 所有的负数都简单地被压缩到零

32:15 so the positive numbers are a path through but everything negative is just set to zero
积极数字是一条通路,但所有负数都设定为零

32:20 and because you are basically throwing away half of the distribution they find in their analysis
并且因为你基本上扔掉了一半的分布, 他们的分析发现他们有一半的分布

distribution
分布 分布
数学与微积分

32:25 of the forward activations in the neural net that you have to compensate for that with a gain
神经网中的前置激活,您必须用收益来补偿

neural net
神经网络 神经网络
神经网络基础

32:32 and so here they find that basically when they initialize their weights they have to do it with a
所以他们发现,基本上当他们开始 他们的权重时, 他们必须做一个

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

32:37 zero mean Gaussian whose standard deviation is square root of 2 over the fan in what we have here
标准偏差为平方根的高斯人 2 位于我们这里的风扇上

32:44 is we are initializing the Gaussian with the square root of fan in this and all here is the fan in
我们是在用扇子的平方根 开始高西亚人的首字母首字母首版,这里的都是

32:50 so what we have is square root of 1 over the fan in because we have the division here
因为我们这里有分局

32:56 now they have to add this factor of 2 because of the ReLU which basically discards half of the
现在他们必须加上2的这个因数,因为RELU基本上抛弃了一半

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

33:01 distribution and clamps it at zero and so that's where you get an initial factor
分配分布,然后在零处将其夹紧,所以这就是你获得初始因数的地方

distribution
分布 分布
数学与微积分

33:06 now in addition to that this paper also studies not just the sort of behavior of the activations
除了现在的这个文件之外, 本文还不仅研究 激活的行为类型

33:12 in the forward pass of the neural net but it also studies the back propagation and we have to
神经网的前向传承 但它研究后传播 我们必须

forward pass
前向传播 前向传播
神经网络基础

neural net
神经网络 神经网络
神经网络基础

33:16 make sure that the gradients also are well behaved and so because ultimately they end
确保梯度也保持良好行为,因为梯度最终会结束

33:22 up updating our parameters and what they find here through a lot of the analysis that I
更新我们的参数 通过我所做的很多分析 他们在这里发现什么

33:28 am much to read through but it's not exactly approachable what they find is basically
读读读读的很多,但并不完全适合他们

33:33 if you properly initialize the forward pass the backward pass is also approximately initialized
如果您正确初始化了前方过道, 后向过往的过往也大致初始化

forward pass
前向传播 前向传播
神经网络基础

backward pass
反向传播 反向传播
神经网络基础

33:39 up to a constant factor that has to do with the size of the number of hidden neurons in an
与隐藏神经元数量大小有关的恒定系数

33:45 early and late layer and but basically they find empirically that this is not a choice that matters
早期和末层,但基本上他们从经验中发现 这不是一个重要的选择

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

33:52 too much now this climbing initialization is also implemented in PyTorch so if you go to torch.innet
皮托尔奇也实施了这种攀爬首创,

pytorch
PyTorch PyTorch
代码与工程

33:59 documentation you'll find climbing normal and in my opinion this is probably the most common
文档会发现攀爬正常 在我看来这大概是最常见的

34:04 way of initializing neural networks now and it takes a few keyword arguments here so another
开始启动神经网络的方式 现在,它需要一些关键词参数 这里这样另一个

34:10 one it wants to know the mode would you like to normalize the activations or would you like to
想要知道模式 您是否愿意让启动程序正常化 或者您愿意

34:15 normalize the gradients to to be always Gaussian with zero mean and a unit or one standard deviation
将梯度正常化,使渐变始终为零平均值高西安,并有一个单位或一个标准差

34:22 and because they find the paper that this doesn't matter too much most of the people just leave
因为他们发现报纸上说 这不重要 大多数人都离开

34:25 it as the default which is fan in and then second pass in the nonlinearity that you are using
是一个默认值, 它在使用的非线性中扇扇和第二遍

34:31 because depending on the nonlinearity we need to calculate a slightly different gain
因为取决于非线性, 我们需要计算一个稍有不同收益

34:35 and so if your nonlinearity is just linear so there's no nonlinearity then the gain here will be one
所以如果你的无线性只是线性的, 那么没有非线性,那么这里的收益将是

34:42 and we have the exact same kind of formula that we've got up here but if the nonlinearity is
我们的公式和我们这里的公式完全一样 但如果非线性是

34:47 something else we're going to get a slightly different gain and so if we come up here to
其它的,我们将会得到一个稍有不同的好处 所以如果我们来这里

34:50 the top we see that for example in the case of ReLU this gain is a square root of 2 and
上面的我们可以看到,例如,在雷卢特的情况下,这一收益是2和2平方根。

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

34:56 the reason it's a square root because in this paper you see how the 2 is inside of the square
是因为在这张报纸上 你看到2是方块里面的

35:05 root so the gain is a square root of 2 in the case of linear or identity we just get a gain of one
根根,所以收益是平方根 2的二 在线性或身份的情况下 我们只得到一个

35:13 in the case of 10h which is what we're using here the advised gain is a 5 over 3 and intuitively
10小时的情况,这就是我们在这里使用 我们的忠告收益是5+3 直觉上

35:19 why do we need a gain on top of the initialization is because 10h just like ReLU is a contractive
为什么我们需要在启动前 得到好处 因为10小时就像雷路一样

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

35:26 transformation so what that means is you're taking the output distribution from this matrix
这意味着你正在从这个矩阵中取出输出分布

distribution
分布 分布
数学与微积分

matrix
矩阵 矩阵
数学与微积分

35:30 multiplication and then you are squashing it in some way now ReLU squashes it by taking
乘法,然后你再用某种方式压压它 现在雷路通过采取

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

35:35 everything below zero and clamping it to zero 10h also squashes it because it's a contractive operation
将零下的一切 压到零10小时 也压扁它,因为它是缩缩手术

operation
运算 运算
神经网络基础

35:40 it will take the tails and it will squeeze them in and so in order to fight the squeezing in
它将抓住尾巴,然后挤压它们,这样就能打住挤压的尾巴。

35:46 we need to boost the weights a little bit so that we renormalize everything back to standard unit
我们需要提高重量, 以便重新整顿所有的东西 回到标准单位

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

35:51 standard deviation so that's why there's a little bit of a gain that comes out now i'm skipping
所以现在我跳过

35:57 through this section a little bit quickly and i'm doing that actually intentionally and the reason
透过这段内容,我很快地 做了一些故意的

36:01 for that is because about seven years ago when this paper was written you had to actually be extremely
这是因为大约七年前,当这份文件写成时 你实际上必须非常

Adam 优化器 · 梯度下降法 36:07

36:07 careful with the activations and the gradients and their ranges and their histograms and you
注意电动和梯度 以及它们的射程 直方图 还有你

36:11 have to be very careful with the precise setting of gains and the scrutinizing of the non-linearities
必须非常小心地准确确定收益的准确定值和仔细审查非线性

36:15 used and so on and everything was very finicky and very fragile and to be very properly arranged
使用等等,一切都非常虚弱脆弱,安排得非常妥当

36:21 for the neural net to train especially if your neural net was very deep but there are a number
神经网需要训练 尤其是如果神经网很深 但是有数

neural net
神经网络 神经网络
神经网络基础

36:25 of modern innovations that have made everything significantly more stable and more well behaved
现代创新使一切变得更加稳定、行为更加良好

36:29 and it's become less important to initialize these networks exactly right and some of those modern
开始建立这些网络已经变得不重要了 准确正确,一些现代的

36:34 innovations for example are residual connections which we will cover in the future the use of a
例如,我们今后将使用一个

36:39 number of normalization layers like for example batch normalization layer normalization group
诸如批次等正常化层的标准化层数目

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

batch normalization
批归一化 批归一化
训练与优化

36:46 normalization we're going to go into a lot of these as well and number three much better
正常化,我们要去进入 很多这些和 这些以及第三更好的

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

36:49 optimizers not just to cast a gradient descent the simple optimizer we're basically using here
优化器,不只是用来给平滑下降 我们在这里基本上使用的简单优化器

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

36:55 but slightly more complex optimizers like rms prop and especially adam and so all of these modern
更复杂一些的优化剂 比如RM道具,特别是Adam

adam
Adam 优化器 Adam 优化器
训练与优化

37:00 innovations make it less important for you to precisely calibrate the initialization of the
使您不太需要精确校准

37:04 neural net all that being said in practice what should we do in practice when i initialize these
在实践中所说的所有实际中我们实际上应该做些什么,当我开始这些时

neural net
神经网络 神经网络
神经网络基础

37:10 neural nets i basically just normalize my weights by the square root of the fan in so
基本上我刚刚将重量 正常化,在扇子的平根

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

37:16 basically roughly what we did here is what i do now if we want to be exactly accurate here
基本上我们在这里所做的是 我现在要做的 如果我们想准确的说

37:22 we and go back in it of uh climbing normal this is how it would be implemented we want to set the
我们回到正常的攀爬中 这样才能执行 我们要设置

37:30 standard deviation to be gained over the square root of fan in right so to set the standard deviation
将右侧扇扇根平根上取得的标准差以设定标准差

37:37 of our weights we will proceed as follows basically when we have torshtrad ran in and let's
等我们有了磨衣的进进进,我们接下来要开始

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

37:43 say i just create a thousand numbers we can look at the standard deviation of this and of course
我只是说,我只是创造一千个数字 我们可以看这个和当然

37:47 that's one that's the amount of spread let's make this a bit bigger so it's closer to one
也就是扩散量的分布,让我们把它放大一点 这样它更接近一个

37:52 so that's the spread of the Gaussian of zero mean and unit standard deviation now basically when you
这就是高西亚零平均差和单位标准差的分布, 基本上当你

37:58 take these and you multiply by say point two that basically scales down the Gaussian and that makes
以这些为取,然后乘以第二点 基本按高山的大小计算

38:04 its standard deviation point two so basically the number that you multiply by here ends up being
其标准偏差点2, 基本是您在此乘以的数值, 结果最终

38:09 the standard deviation of this Gaussian so here this is a standard deviation point two Gaussian
此高斯的标准偏差, 因此这里是标准偏差点 2 高西亚

38:16 here when we sample our w1 but we want to set the standard deviation to gain over square root of fan
当我们在这里抽样我们的 w1 时, 我们在此进行此样本, 但我们想要设置标准偏差以获取超过平方根的扇扇形

sample
样本 样本
神经网络基础

38:23 mode which is fan in so in other words we want to multiply by gain which for 10h is 5 over 3
模式,它以扇扇方式在其中,换句话说,我们想乘以收益,10小时为5+3

38:32 5 over 3 is the gain and then times um i guess divide uh square root of the fan in and in this example
5+3是收益,然后是平方根

38:52 here the fan in was 10 and i just noticed that actually here the fan in for w1 is actually an
风扇在10,我刚刚注意到 实际上这里的W1的风扇

38:58 embed times block size which as you will recall is actually 30 and that's because each character
嵌入区块大小, 您会记得, 实际为30, 因为每个字符

39:03 is 10 dimensional but then we have three of them and we concatenate them so actually the fan in
10维是10维 但是我们有三个,我们把它们结合成

39:06 here was 30 and i should have used 30 here probably but basically we want 30 uh square root so this is
本来我应该用三十块的 但是基本上我们需要30平方根 所以这是

39:13 the number this is what our standard deviation we want to be and this number turns out to be point
数字,这是我们想要的标准偏离,这个数字是点

39:18 three whereas here just by fiddling with it and looking at the distribution and making sure it
三个,在这里只是通过 乱动它,看分布和确保它

distribution
分布 分布
数学与微积分

39:22 looks okay uh we came up with point two and so instead what we want to do here is we want to
我们想出了第二点,我们想来 而不是我们想在这里做的

39:28 make the standard deviation be um 5 over 3 which is our gain divide this amount times point two
标准差是 um 5 乘 3 等于 3 = = 3 。

39:40 square root and these brackets here are not that uh necessary but i'll just put them here for clarity
方根和这些括号没有那么必要 但我会放在这里,清楚一点

39:46 this is basically what we want this is the kimi in it in our case for a 10h nonlinearity
基本上这就是我们想要的, 也就是我们的情况中的 Kimi 和10小时不线性

39:52 and this is how we would initialize the neural net and so we're multiplying by point three
这就是我们如何启动神经网的初始化 所以我们乘以第三点

neural net
神经网络 神经网络
神经网络基础

39:57 instead of multiplying by point two and so we can we can initialize this way and then we can
而不是乘以点2,这样我们就可以开始这样的方式 然后我们可以

40:05 train the neural net and see what we got okay so i trained the neural net and we end up in roughly
训练神经网,然后我们开始研究 我们得到了什么好 所以我训练了神经网

neural net
神经网络 神经网络
神经网络基础

40:11 the same spot so looking at the validation loss we now get 2.10 and previously we also had 2.10
同一地点,我们查看了验证损失,现在我们得到2.10美元,以前我们也有2.10美元。

validation
验证 验证
训练与优化

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

40:17 there's a little bit of a difference but that's just the randomness of the process i suspect
有点不同,但这只是我怀疑过程的随机性

40:21 but the big deal of course is we get to the same spot but we did not have to introduce any
但最重要的当然是 我们到了同一个地方 但我们没有介绍

40:26 um magic numbers that we got from just looking at histograms and guessing checking we have something
um mamm magic number 我们能从直方图和猜测中得到的 微小数字

40:32 that is semi-principled and will scale us to much bigger networks and something that we can
将把我们的规模扩大到更大的网络和我们能够实现的

40:38 sort of use as a guide so i mentioned that the precise setting of these initializations is
用作指南的某类用途指南, 所以我提到这些初始化的精确设置是

40:43 not as important today due to some modern innovations and i think now is a pretty good
今天由于现代创新,没有那么重要。 我认为现在是一个不错的

40:46 time to introduce one of those modern innovations and that is batch normalization
时间来引进其中一种现代创新,即批次正常化

batch normalization
批归一化 批归一化
训练与优化

40:51 so batch normalization came out in uh 2015 from a team at google and it was an extremely impactful
所以批量正常化 2015年 uh 从谷歌的一个团队里出来

batch normalization
批归一化 批归一化
训练与优化

40:57 paper because it made it possible to train very deep neural nets quite reliably and it basically
因为它使得能够非常可靠地 训练深心神经网

验证 · 缩放 41:03

41:03 just worked so here's what batch normalization does and let's implement it um basically we have
刚工作,所以这里的批量正常化 和我们实施它

batch normalization
批归一化 批归一化
训练与优化

41:10 these hidden states hpreact right and we were talking about how we don't want these uh these
这些隐藏状态 预示正确 我们正讨论我们如何不想要这些

41:17 pre activation states to be way too small because the than the 10h is not doing anything
启动前激活状态表示太小, 因为十小时以上没有做任何事情

activation
激活值 激活值
神经网络基础

41:23 but we don't want them to be too large because then the 10h is saturated in fact we want them to be
但我们不希望它们太大 因为10小时饱和了 事实上我们想要它们

41:28 roughly roughly Gaussian so zero mean and a unit or a one standard deviation at least
大致大致上高西亚为零平均值,至少一个单位或一个标准差

41:34 initialization so the insight from the batch normalization paper is okay you have these
初始化,这样批量正常化文件的洞察力 就可以了

batch normalization
批归一化 批归一化
训练与优化

41:39 hidden states and you'd like them to be roughly Gaussian then why not take the hidden states
隐藏状态,你希望它们大致是高山的 为何不取隐藏状态

41:45 and just normalize them to be Gaussian and it sounds kind of crazy but you can just do that because
并且只是让他们正常化 他们高西亚,听起来有点疯狂 但是你可以这样做 因为

41:53 standardizing hidden states so that their unit Gaussian is a perfectly differentiable operation
使隐藏状态标准化,使其单位高森是一个完全不同的操作

operation
运算 运算
神经网络基础

41:58 as we'll soon see and so that was kind of like the big insight in this paper and when I first
很快我们就会看到,所以这有点像 这文件中的深刻见解 当我第一次

42:02 read it my mind was blown because you can just normalize these hidden states and if you'd
读到它,我脑中被吹了 因为你可以只是正常化 这些隐藏状态 如果你愿意

42:06 like unit Gaussian states in your network uh at least initialization you can just normalize
单位像高西亚州一样 在网络中的状态 呃 至少初始化您可以正常

42:12 them to be unit Gaussian so uh let's see how that works so we're going to scroll to our
高森部队,让我们看看如何运作 这样我们就可以滚到

42:17 pre activations here just before they enter into the 10h now the idea again is remember we're trying
刚刚进入10点前的预启动程序 现在又想记住 我们在尝试

42:22 to make these roughly Gaussian and that's because if these are way too small numbers then the 10h
做这些粗略的高西亚人, 因为如果这些数字太小的话,那么10小时

42:28 here is kind of inactive but if these are very large numbers then the 10h is way too saturated
这里有点不活动, 但如果数量非常多, 那么10小时是饱和的太远

42:34 and greatest in the flow so we'd like this to be roughly Gaussian so the insight in
最伟大的流动,所以我们希望这 大约是高山的

42:40 batch normalization again is that we can just standardize these activations so they are exactly
批量正常化是,我们只能 标准化这些启动装置,所以它们完全

batch normalization
批归一化 批归一化
训练与优化

42:45 Gaussian so here hpreact has a shape of 32 by 200 32 examples by 200 neurons in the hidden layer
Gaussian 所以这里的突变形状 以32乘200 32 示例乘200个神经元 在隐藏层中

hidden layer
隐藏层 神经网络 中的合成层，介于输入层 （即特征）和输出层 （即预测）之间。神经网络包含一个或多个隐藏层。
神经网络基础

42:55 so basically what we can do is we can take hpreact and we can just calculate the mean
所以我们基本上可以做的是 我们可以采取预想,我们可以计算出

43:01 and the mean we want to calculate across the zero dimension and we want to also keep them as
以及我们想要计算出在零维的平均值, 我们也想把它们保留为

43:07 true so that we can easily broadcast this so the shape of this is 1 by 200 in other words we are
以便我们能够轻松地播放这个节目, 也就是说,我们的形状是1乘200

43:15 doing the mean over all the uh elements in the batch and similarly we can calculate the standard
做这个意思 在批量中的所有uh元素上 类似地 我们可以计算标准

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

43:23 deviation of these activations and that will also be 1 by 200 now in this paper they have the
和在本文件中现在的1x200,

43:31 uh sort of prescription here and see here we are calculating the mean which is just taking
额,呃,一种处方 在这里和看这里 我们计算平均值 就是采取

43:38 the average value of any neurons activation and then the standard deviation is basically kind of like
任何神经元激活的平均值 然后标准偏差基本上就差不多

activation
激活值 激活值
神经网络基础

43:46 um the measure of the spread that we've been using which is the distance of every one of these
um 测量我们一直使用的分布度 也就是每一个这些

43:53 values away from the mean and that squared and averaged that's the that's the variance and then
值的差值,然后是平方和平均的, 这是差异 然后

44:01 if you want to take the standard deviation you would square root the variance to get the standard
如果您想要选择标准差, 您将会平方根差异以获得标准

44:06 deviation so these are the two that we're calculating and now we're going to normalize
偏移,所以这是我们计算中的两个 现在我们要恢复正常

44:11 or standardize these x's by subtracting the mean and um dividing by the standard deviation
或 将这些 x 的值标准化, 减去平均值和平方除以标准差

44:17 so basically we're taking nth preact and we subtract the mean and then we divide by the standard
所以我们基本上采取nth 预言,我们减去平均值 然后我们按照标准划分

44:31 deviation this is exactly what these two std and mean are calculating oops sorry this is the mean
偏移正值就是这两个标准和平均值 计算ops 对不起,这是这个标准

44:41 and this is the variance you see how the sigma is the standard deviation usually so this is sigma
这就是差异,您可以看到 标准偏差是何等的西格玛, 通常就是此方格玛

44:45 square which the variance is the square of the standard deviation so this is how you standardize
方方方方中,差异的方位是标准差的平方,因此这就是如何使标准差标准化

44:52 these values and what this will do is that every single neuron now and its firing rate will be
以及它的发射速度

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

44:57 exactly unit Gaussian on these 32 examples at least of this batch that's why it's called batch
确切单位高西亚 以这32个例子 至少这批中至少有这批

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

45:02 normalization we are normalizing these batches and then we could in principle train this
正在将这些批次正常化,然后原则上可以培训

normalization [ˌnɔːməlaiˈzeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

45:09 notice that calculating the mean and your standard deviation these are just mathematical
计算平均值和标准偏差 这些只是数学

45:13 formulas they're perfectly differentiable although this is perfectly differentiable and we can just
公式是完全不同的 虽然这是完全不同的 我们可以

45:17 train this the problem is you actually won't achieve a very good result with this and the reason for that
问题在于你实际上不会以这个和原因 取得一个很好的结果

45:24 is we want these to be roughly Gaussian but only at initialization but we don't want these to be
我们希望这些大致是高西亚语,但只是在初始化时, 但我们不希望这些

45:31 forced to be Gaussian always we would like to allow the neural net to move this around
总是我们想让神经网 移动这个

neural net
神经网络 神经网络
神经网络基础

45:37 to potentially make it more diffuse to make it more sharp to make some 10 h neurons maybe
有可能让它更扩散 使之变得更锋利 造出大约10个神经元

45:42 mean more trigger more trigger happy or less trigger happy so we'd like this distribution to move around
更能触发更多 更能引发更多 快乐或更少触发 快乐 所以我们希望这个分布

distribution
分布 分布
数学与微积分

45:47 and we'd like the back propagation to tell us how the distribution should move around
我们希望背面传播 告诉我们如何分配应该移动

back propagation
反向传播 反向传播
神经网络基础

distribution
分布 分布
数学与微积分

45:52 and so in addition to this idea of standardizing the activations at any point in the network
并且除了在网络的任何时候使启动标准化的想法之外

45:59 we have to also introduce this additional component in the paper here described as scale and shift
我们还必须在本文中介绍这一额外的构成部分,这里称为规模和变化

46:05 and so basically what we're doing is we're taking these normalized inputs and we are
所以基本上我们要做的就是 把这些正常的投入

46:09 additionally scaling them by some gain and offsetting them by some bias to get our final
以某种收益来补充,并用某些偏见来抵消这些利润,以便最终确定

scaling ['skeiliŋ]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

46:15 output from this layer and so what that amounts to is the following we are going to allow a
此层输出的输出, 因此这个层次的输出结果数量是以下的, 我们将会允许

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

46:21 batch normalization gain to be initialized at just a once and the once will be in the shape of
分批正常化的收益,仅一次启动一次,一次将形成

batch normalization
批归一化 批归一化
训练与优化

46:29 one by n hidden and then we also will have a b and bias which will be torched at zeros
将有一个b和偏差 被点燃在零

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

46:37 and it will also be of the shape and by one by and hidden and then here the bn gain will multiply this
并且成形,一个成形, 一个成形, 隐藏, 然后在这里,bn收益将乘以这个

46:47 and the bn bias will offset it here so because this is initialized to one and this to zero
bn 偏差在此抵消, 因为这个首首首为 1, 零

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

46:54 at initialization each neurons firing values in this batch will be exactly unit Gaussian
初始化时,每批神经元的发烧值将完全为单位 Gaussian

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

47:01 and will have nice numbers no matter what the distribution of the hp act is coming in
并且将拥有一个不错的数字 无论Hp动作的分布如何

distribution
分布 分布
数学与微积分

47:07 coming out it will be unit Gaussian for each neuron and that's roughly what we want at least at
出来后,它将是单位高森 对于每个神经元, 这大概是 我们想要的 至少

neuron ['njuərɔn]
神经元 神经网络 中的节点，通常会接收多个输入值并生成一个输出值。神经元通过 将激活函数 （非线性转换）应用于输入值的加权和来计算输出值。
神经网络基础

47:11 initialization and then during optimization we'll be able to back propagate to be and gain
初始化,然后在优化时 我们可以重新传播 以获得

47:17 and be in bias and change them so the network is given the full ability to
并且偏见,改变它们,使网络完全有能力

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

47:21 do with this whatever it wants internally here we just have to make sure that we
做这个,不管它想做什么 在这里内部我们只需要确保

47:28 um include these in the parameters of the neural nut because they will be trained with back propagation
um 包括这些内容,以纳入神经神经螺旋参数,因为将对他们进行背传播培训。

back propagation
反向传播 反向传播
神经网络基础

47:35 so let's initialize this and then we should be able to train and then we're going to also
所以先开始这个,然后我们就可以训练 然后我们也可以

47:47 copy this line which is the batch normalization layer here on a single line of code and we're
复制此行, 这是批量正常化层 在单行代码的一行

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

47:53 going to swing down here and we're also going to do the exact same thing at test time here
准备在这里摇摆,我们也会做 同样的事情 在这里的测试时间

48:01 so similar to train time we're going to normalize and then scale and that's going to give us our
类似训练时间,我们要去正常化 然后规模,这将给我们

48:07 train and validation loss and we'll see in a second that we're actually going to change this a
火车和验证损失 等一秒后我们就会看到 我们实际上将改变

validation
验证 验证
训练与优化

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

48:12 little bit but for now i'm going to keep it this way so i'm just going to wait for this to
一点点,但现在我要保持这样 所以我要等待这个

48:16 converge okay so i allowed the neural nut to converge here and when we scroll down we see that
集中到一起,所以我让神经螺母聚集在这里 当我们滚下来时,我们看到

48:21 our validation loss here is 2.10 roughly which i wrote down here and we see that this is
验证损失大约2.10美元 我写下来,我们发现

validation
验证 验证
训练与优化

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

48:27 actually kind of comparable to some of the results that we've achieved previously now i'm not actually
和我们以前取得的一些结果相比 我其实不是

48:32 expecting an improvement in this case and that's because we are dealing with a very simple neural
因为我们正面对一个非常简单的神经

残差网络 · 卷积层 48:36

48:36 nut that has just a single hidden layer so in fact in this very simple case of just one hidden layer
坚果只有一层隐藏层 所以事实上,在这个非常简单的 仅一个隐藏层的情况下

hidden layer
隐藏层 神经网络 中的合成层，介于输入层 （即特征）和输出层 （即预测）之间。神经网络包含一个或多个隐藏层。
神经网络基础

48:42 we were able to actually calculate what the scale of w should be to make these pre-activations
我们实际上能够计算出 使这些预激活的 规模应该有多大

48:48 already have a roughly Gaussian shape so the batch normalization is not doing much here
已经大致有高西亚形状 所以批量正常化在这里没有做什么

batch normalization
批归一化 批归一化
训练与优化

48:53 but you might imagine that once you have a much deeper neural nut that has lots of different types
但可以想象 一旦你有一个更深得多的神经螺旋 有很多不同的类型

48:57 of operations and there's also for example residual connections which we'll cover and so on it will
行动,还有剩余连接 将覆盖,等等

49:03 become basically very very difficult to tune the scales of your weight matrices such that all
基本上很难调整你体重矩阵的尺度,以便

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

49:09 the activations throughout the neural nut are roughly Gaussian and so that's going to become
整个神经神经螺管的激活 大约是高森,所以这将会变成

49:14 very quickly intractable but compared to that it's going to be much much easier to sprinkle
很快的难以解决 但相比于它, 它会更容易撒洒

49:19 batch normalization layers throughout the neural nut so in particular it's common to look at every
神经神经螺管的分批正常化层, 所以特别常见于

batch normalization
批归一化 批归一化
训练与优化

49:25 single linear layer like this one this is a linear layer multiplying by weight matrix and adding the
以重量矩阵乘以并添加

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

49:29 bias or for example convolutions which we'll cover later and also perform basically a
或例如,我们稍后会覆盖的 并基本上

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

49:35 multiplication with the weight matrix but in a more spatially structured format it's custom
与加权矩阵乘法,但以空间结构化格式,是自定义

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

matrix
矩阵 矩阵
数学与微积分

49:40 it's customary to take these linear layer or convolutional layer and append a batch
通常都用线性层或卷变层来附加批次

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

convolutional layer
卷积层 深度神经网络的一个层，卷积过滤器 会在其中传递输入矩阵。以下面的 3x3 卷积过滤器 为例： 下面的动画显示了一个由 9 个卷积运算（涉及 5x5 输入矩阵）组成的卷积层。请注意，每个 卷积运算都涉及一个不同的 3x3 输入矩阵切片。由此产生的 3×3 矩阵（右侧）就包含 9 个卷 积运算的结果：
神经网络基础

49:45 normalization layer right after it to control the scale of these activations at every point in the
之后的正常化层 控制这些激活的规模 在

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

49:50 neural nut so we'd be adding these batch normal layers throughout the neural nut and then this
神经神经神经神经神经,所以我们就会在神经神经神经中 添加这些正常的分批层

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

49:55 controls the scale of these activations throughout the neural nut it doesn't require us to do
控制这些激活的大小 在整个神经神经螺管上 它不需要我们做

50:00 perfect mathematics and care about the activation distributions for all these different types of
注意所有不同类型

activation
激活值 激活值
神经网络基础

50:05 neural network uh lego building blocks that you might want to introduce into your neural nut
神经网络 legeo 构件 你可能想引入

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

50:09 and it significantly stabilizes the train and that's why these layers are quite popular
并且它大大稳定了火车的稳定性 这就是为什么这些层子很受欢迎

50:14 now the stability offered by batch normalization actually comes at a terrible cost and that cost
现在批量正常化所提供的稳定 以可怕的代价

batch normalization
批归一化 批归一化
训练与优化

50:19 is that if you think about what's happening here something something terribly strange and unnatural
如果你想想这里发生的事 某些奇怪和不正常的事情

50:24 is happening it used to be that we have a single example feeding into a neural nut and then uh we
发生之前我们有一个例子 输入神经神经,然后...

50:31 calculated this activations and its logits and this is a deterministic sort of process so you
计算此激活及其登录, 这是一个决定性的过程类型, 所以您可以

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

50:37 arrive at some logits for this example and then because of efficiency of training we suddenly
突然间我们为了训练效率 突然之间

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

50:42 started to use batches of examples but those batches of examples were processed independently
开始使用批量实例,但这些批次实例是独立处理的

50:47 and it was just an efficiency thing but now suddenly in batch normalization because of the
并且这只是效率问题, 但现在突然在批量正常化

batch normalization
批归一化 批归一化
训练与优化

50:51 normalization through the batch we are coupling these examples mathematically and in the forward
通过分批的正常化 我们用数学和前方 将这些例子结合起来

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

50:57 pass and the backward pass of a neural net so now the hidden state activations hpact and your
和神经网的后向传递, 所以现在隐藏状态激活 hpcott 和你

backward pass
反向传播 反向传播
神经网络基础

neural net
神经网络 神经网络
神经网络基础

51:03 logits for any one input example are not just a function of that example and its input
任何一个输入示例的日志记录不仅仅是该示例及其输入的函数

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

51:08 but they're also a function of all the other examples that happen to come for a ride
但它们也是所有其他例子的函数 碰巧来兜风

51:12 in that batch and these examples are sampled randomly and so what's happening is for example
以批次中的那批中,这些例子是随机抽样的, 所以现在的情况就是例如

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

51:17 when you look at hpact that's going to feed into h the hidden state activations for for example for
当您查看 hpcat 时, 它会输入h 隐藏状态激活, 例如

51:23 for any one of these input examples is going to actually change slightly depending on what other
对于其中任何一个输入示例, 任何输入示例实际上都会根据其他

51:28 examples there are in the batch and and depending on what other examples happen to come for a ride
批量中存在实例,并取决于其他偶然出现的情况

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

51:34 h is going to change suddenly and it's going to like jitter if you imagine sampling different
这会喜欢紧张 如果你想象不同取样

51:38 examples because the the statistics of the mean and the standard deviation are going to be impacted
举例,因为中值和标准偏差的统计将受到影响

51:43 and so you'll get a jitter for h and you'll get a jitter for logits
这样你就会紧张起来,然后你也会紧张起来

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

51:48 and you think that this would be a bug or something undesirable but in a very strange
并且你认为这将是一个虫子 或者一些不可取的东西 但在一个非常奇怪的

51:53 way this actually turns out to be good in neural network training and as a side effect
这样做实际上证明在神经网络训练方面是好的,并且是一种副作用

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层 ），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

51:59 and the reason for that is that you can think of this as kind of like a regularizer
原因在于你可以把这当成一个正规化剂

52:03 because what's happening is you have your input and you get your h and then depending on the other
因为现在发生的事情是 你有你的投入 得到你的h 然后取决于其他

52:07 examples this is jittering a bit and so what that does is that it's effectively padding out
举例来说,这是有点乱,所以,这实际上就是它实际上被排出

52:12 any one of these input examples and it's introducing a little bit of entropy and
任何其中一种输入的例子 它正在引入一点点的 entropy,

entropy
熵 熵
数学与微积分

52:17 because of the padding out it's actually kind of like a form of data augmentation which will
因为它实际上是一种数据增强的形式 它会

52:21 cover in the future and it's like kind of like augmenting the input a little bit and jittering
未来盖子的覆盖 就像是加点输入 一点点和抖

52:26 it and that makes it harder for the neural nets to overfit these concrete specific examples so by
这使得神经网更难于 过度适应这些具体的具体例子

52:32 introducing all this noise it actually like pads out the examples and it regularizes the neural net
引入所有这些噪音 它实际上喜欢垫板 出示例和规范神经网

neural net
神经网络 神经网络
神经网络基础

52:37 and that's one of the reasons why deceivingly as a second order effect this is actually a regularizer
这就是为什么欺骗作为第二顺序效果的原因之一 实际上,这是个常规化器

52:43 and that has made it harder for us to remove the use of vaturalization because basically no one
这使得我们更难取消使用人工化,因为基本上没有人

52:49 likes this property that the the examples in the batch are coupled mathematically and in the
将批中示例的数学和

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

52:54 forward pass and at least all kinds of like strange results will go into some of that in a second as
至少在各种类似奇怪的结果中, 都会在一秒钟内

forward pass
前向传播 前向传播
神经网络基础

53:00 well and at least a lot of bugs and and so on and so no one likes this property and so people have
至少有很多虫子等等, 没有人喜欢这块地 所以人们有

53:07 tried to deprecate the use of batch normalization and move to other normalization techniques
试图放弃使用批次正常化,并采用其他正常化技术

batch normalization
批归一化 批归一化
训练与优化

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

53:12 that do not couple the examples of a batch examples are layer normalization instance
分批例子的例子与分批实例不相比的是分层正常化实例。

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

53:17 normalization group normalization and so on and we'll come we'll come or some of these later
正常集团正常化等等 我们来我们一起来,或者晚些再过来

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

53:23 but basically long story short batch normalization was the first kind of normalization
但基本上长的短 批量正常化 第一种正常化

batch normalization
批归一化 批归一化
训练与优化

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

53:27 layer to be introduced it worked extremely well it happens to have this regularizing effect
使用该程序非常有效, 碰巧具有这种正规化效果

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

53:33 it stabilized training and people have been trying to remove it and move to some of the
培训稳定下来,人们一直在试图清除这些训练,并搬往一些

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

53:38 other normalization techniques but it's been hard because it just works quite well
其它正常化技术 但一直以来都很难 因为效果很好

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

53:44 and some of the reason that it works quite well is again because of this regularizing effect
之所以效果很好,原因之一 又是因为这个常规化的效果

53:48 and because of the because it is quite effective at controlling the activations and their distributions
这是因为它控制激活及其分布相当有效

53:54 so that's kind of like the brief story of batch normalization and I'd like to show you
有点像批量正常化的简单故事 我想给你们看

batch normalization
批归一化 批归一化
训练与优化

53:58 one of the other weird sort of outcomes of this coupling so here's one of the strange
另一件奇怪的结果之一 这样的结合 所以这里是一个奇怪的

54:04 outcomes that I only glossed over previously when I was evaluating the loss on the validation
之前我对确认损失进行评估时 所掩盖的结果

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

validation
验证 验证
训练与优化

54:09 set basically once we've trained a neural net we'd like to deploy it in some kind of a setting
基本上一旦我们训练了神经网 我们想在某种环境下部署它

neural net
神经网络 神经网络
神经网络基础

54:15 and we'd like to be able to feed in a single individual example and get a prediction out from
我们希望能够以单个的个体例子 进行喂食,并从

54:19 our neural net but how do we do that when our neural net now in the forward pass estimates the
但是当我们神经网现在进入前传估计时 我们如何做到这一点

neural net
神经网络 神经网络
神经网络基础

forward pass
前向传播 前向传播
神经网络基础

54:25 statistics of the mean understanding deviation of a batch the neural net expects batches as an input
神经网预期批量作为一种投入的批量平均理解偏差

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

neural net
神经网络 神经网络
神经网络基础

54:29 now so how do we feed in a single example and get sensible results out and so the
现在,我们如何用一个例子来说明我们是如何进食的,怎样才能得出合理的结果。

54:35 proposal in the batch normalization paper is the following what we would like to do here
批次正常化文件中的建议是,我们在此希望做的如下:

batch normalization
批归一化 批归一化
训练与优化

54:40 is we would like to basically have a step after training that calculates and sets the batch
基本上我们想在训练后再有一个步骤 来计算和设置批量

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

54:47 normal mean and certain deviation a single time over the training set and so I wrote this code here
和某些偏差 训练时间的单一时间 所以,我在这里写了这个代码

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

54:53 in interest of time and we're going to call what's called calibrate the batch normal statistics
时间的利害关系,我们称它为校准批量正常统计

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

54:59 and basically what we do is torsion torsion no grad telling by torch that none of this we
基本上我们所做的就是 火炬不给格勒说 我们没有这些

55:04 will call the dot backward on and it's going to be a bit more efficient we're going to
就会叫点后退,它会更有效率 我们将会去

55:09 take the training set get the preactivations for every single training example and then one
参加培训后,每个培训实例的预激活时间都将在培训中进行,然后是

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

55:14 single time estimate the mean and standard deviation over the entire training set and then we're going
单时间估计整个训练的平均值和标准差 然后我们开始

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

55:18 to get b and mean and b and standard deviation and now these are fixed numbers estimated over the
以获得 b和平均值,b 和标准差,现在这些是估计的固定数字。

55:23 entire training set and here instead of estimating it dynamically we are going to instead here
整个训练组和这里,而不是动态估算,我们在这里要改

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

55:31 use b and mean and here we're just going to use b and standard deviation
使用b和b,然后用b和标准差

55:37 and so at test time we are going to fix these clamp them and use them during inference and now
所以在试验时间 我们要修正这些夹紧 并在推断时使用

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

55:45 you see that we get basically identical result but the benefit that we've gained is that we can
你看,我们得到的结果基本相同 但我们已经获得的好处是 我们可以

55:51 now also forward a single example because the mean and standard deviation are now fixed
现在还举一个例子,因为中值和标准偏离现已固定

55:55 sort of tensors that said nobody actually wants to estimate this mean and standard deviation
说没有人真的想估计这个平均值和标准偏差

56:01 as a second stage after neural network training because everyone is lazy and so this batch normalization
成为神经网络培训的第二阶段,因为每个人都懒惰,因此这批人正常

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

56:07 paper actually introduced one more idea which is that we can we can estimate the mean and standard
纸上实际上还引入了另一个想法, 就是我们可以估算平均值和标准

56:11 deviation in a running matter running manner during training of the neural net and then we can
神经网训练过程中运行时,我们就可以

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

neural net
神经网络 神经网络
神经网络基础

56:18 simply just have a single stage of training and on the side of that training we are estimating
仅仅有一个培训阶段,而我们估计的是在培训的一个单一阶段

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

56:22 the running mean and standard deviation so let's see what that would look like let me basically
运行中平均和标准偏差的运行平均值和标准偏移, 所以让我们来看看什么样子

56:27 take the mean here that we are estimating on the batch and let me call this b and mean on the i
意思是这里我们估计的批量数量 让我把这叫做b

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

56:32 iteration and then here this is b and std b and std i okay and the mean comes here and the std comes
中间线从这里过来,然后这个是b和stdb和stDi

iteration [.itə'reiʃən]
迭代 模型的权重在训练期间的一次更新。迭代包含计算参数在单批次 数据上的梯度损失。 K k-means 一种热门的聚类 算法，用于对非监督式学习中的样本进行分组。 k-means 算法基 本上会执行以下操作： 以迭代方式确定最佳的 k 中心点（称为形心 ）。 将每个样本分配到最近的形心。与同一个形心距离最近的样本属于同一个组。 k-means
神经网络基础

56:52 here so so far i've done nothing i've just moved around and i created these extra variables for
到目前为止我在这里 什么都没做 我刚刚搬来搬去

56:56 the mean and standard deviation and i've put them here so so far nothing has changed but what
和标准偏离 我把它们放在这里 到目前为止,没有任何改变 除了什么

57:01 we're going to do now is we're going to keep a running mean of both of these values during training
我们现在要做的是 训练时保持这两种价值观的运行

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

57:06 so let me swing up here and let me create a bn mean underscore running and i'm going to initialize it at
让我来跳一摇,让我 创建一个BN的意思强调跑步,我准备在

57:14 zeros and then bn std running which i'll initialize at once because in the beginning because of the
零零,然后是bn std 运行,我会同时初始化, 因为开始是因为

57:25 way we initialized w1 and b1 hpact will be roughly unit Gaussian so the mean will be
初始化的 w1 和 b1 hpcott 将大致为单位高西亚, 平均值

57:31 roughly zero and the standard deviation roughly one so i'm going to initialize these that way
大约零,标准差大约是1 所以我要开始这些

57:37 but then here i'm going to update these and in pytorch these mean and standard deviation that are
但在这里我要更新这些 并以燃烧的火炉 来更新那些

pytorch
PyTorch PyTorch
代码与工程

57:44 running they're not actually part of the gradient based optimization we're never going to derive
运行它们实际上不是基于梯度的优化的一部分 我们永远不会得到

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

57:48 gradients with respect to them they're they're updated on the side of training and so what
有关这些梯度的梯度,它们更新了 在训练的侧面,所以

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

57:54 we're going to do here is we're going to say with torshtrap no grad telling pytorch that
我们在这里要做的是 用绞刑套说 格斗不告诉皮托奇

pytorch
PyTorch PyTorch
代码与工程

57:59 the update here is not supposed to be building out a graph because there will be no dot backward
这里的更新不应该是建立图表, 因为没有点向后

58:05 but this running mean is basically going to be 0.999 times the current value
但这个运行平均值基本上 将是当前值的0.999倍

58:13 plus 0.001 times the this value this new mean and in the same way bn std running will be
+0.001乘以此值,这个新平均值和以同样方式 bn std 运行

58:23 mostly what it used to be but it will receive a small update in the direction of what the current
大部分是过去的情况,但现在的情况将稍有更新。

58:32 standard deviation is and as you're seeing here this update is outside and on the side of the
标准偏离是,正如你在这里看到的 这个更新是在外和侧面

58:39 gradient based optimization and it's simply being updated not using gradient descent it's just
基于梯度优化的梯度, 它只是更新 而不是使用梯度下下坡

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

58:44 being updated using a jenki like smooth sort of running mean manner and so while the network
使用Jenki来更新,就像平滑的运行中平均方式一样,在网络运行时如此。

58:54 is training and these pre-activations are sort of changing and shifting around during back propagation
训练是训练,这些预起活动在后期传播期间 变化和移动

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

back propagation
反向传播 反向传播
神经网络基础

59:00 we are keeping track of the typical mean and standard deviation and we're estimating them once
我们追踪典型的平均值和标准偏差 估计一下

59:05 and when i run this now i'm keeping track of this in a running manner and what we're hoping for
当我现在运行这个的时候,我正在 以运行的方式追踪这个过程 以及我们希望的

59:12 of course is that the mean bn mean underscore running and bn mean underscore std are going
当然是,BN的意思是打压跑 和BN的意思打压球

59:17 to be very similar to the ones that we calculated here before and that way we don't need a
和我们之前在这里计算过的非常相似 这样我们就不需要

59:23 second stage because we've sort of combined the two stages and we've put them on the side of each other
第二阶段,因为我们把两个阶段结合起来 把它们放在彼此的一边

59:28 if you want to look at it that way and this is how this is also implemented in the basher
如果您想这样看待它, 并且这是如何在击球机中执行的

59:32 normalization layer in pytorch so during training the exact same thing will happen
火炉的正常化层 所以训练期间 同样的事情也会发生

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

59:38 and then later when you're using inference it will use the estimated running mean of both the
然后再在以后 当你使用引文时 它会使用两种

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

59:44 mean and standard deviation of those hidden states so let's wait for the optimization to converge
和隐藏状态的标准偏差,让我们等待优化的趋同

59:50 and hopefully the running mean and standard deviation are roughly equal to these two
希望运行中平均和标准差 大致等于这两个

59:53 and then we can simply use it here and we don't need this stage of explicit calibration at the end
然后我们就可以在这里简单地使用它 我们不需要这个阶段的清晰校准

59:59 okay so the optimization finished i'll rerun the explicit estimation and then the bn mean
优化完成后我会重新做明确的估计 然后BN的意思

1:00:05 from the explicit estimation is here and bn mean from the running estimation during the
直线估算值此处和 bn 平均值,即从该期间的运行估算值中得出

1:00:12 during the optimization you can see is very very similar it's not identical but it's pretty close
优化期间你可以看到非常相似,但很接近

1:00:19 and in the same way bn std is this and bn std running is this as you can see that once again
和Bn std是相同的,这是和bn std运行是这样的,因为你可以看到 再次

1:00:27 they are fairly similar values not identical but pretty close and so then here instead of bn mean
相当相似的值,它们不相同 但很接近 所以在这里而不是BN平均值

1:00:33 we can use the bn mean running instead of bn std we can use bn std running and hopefully the
我们可以使用 bn 的意思是运行而不是 bn std 我们也可以使用 bn st d 运行,希望

1:00:41 validation loss will not be impacted too much okay so it's basically identical and this way we've
验证损失不会受到太多的影响,好吗? 所以它基本上完全相同,我们这样

validation
验证 验证
训练与优化

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:00:47 eliminated the need for this explicit stage of calibration because we are doing it in line over
消除了这一明确校准阶段的必要性,因为我们正在按顺序进行校准。

1:00:53 here okay so we're almost done with batch normalization there are only two more notes that
批量正常化就快完成了 只有两张便条

batch normalization
批归一化 批归一化
训练与优化

1:00:57 i'd like to make number one i've skipped a discussion over what is this plus epsilon doing
我想做第一位 我跳过一个讨论 讨论这个加欧西隆在做什么

1:01:01 here this epsilon is usually like some small fixed number for example 1e negative 5 by default
这里的 epsilon 通常和一些小固定数字类似, 例如默认情况下 1e 负 5

1:01:07 and what it's doing is that it's basically preventing a division by zero in the case that the
并且它所做的是, 它基本上防止了零分法 在那些

1:01:12 variance over your batch is exactly zero in that case here we'd normally have a division by zero
与您的批量的差值相比, 在此处完全为零, 情况是零, 我们通常会以零为零,

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:01:18 but because of the plus epsilon this is going to become a small number in the denominator instead
但由于加以冰雪,它会变成微小的分母

1:01:23 and things will be more well behaved so feel free to also add a plus epsilon here of a very
并且事情会变得更好, 这样可以自由地 也增加一个加一 Essilon在这里

1:01:28 small number it doesn't actually substantially change the result i'm going to skip it in our case
数量很小,但实际上不会改变结果 我将跳过这个案子

1:01:32 just because this is unlikely to happen in our very simple example here and the second thing
因为我们在这里的简单例子中 不太可能发生这种情况 第二件事

1:01:36 i want you to notice is that we're being wasteful here and it's very subtle but right here where we
是我们在这里浪费了, 而且它非常微妙 但是就在这里,我们

1:01:42 are adding the bias into hpreact these biases now are actually useless because we're adding them to the
将偏见加进这些偏差的预示中 现在实际上已经没用了 因为我们把它们加到

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:01:49 hpreact but then we are calculating the mean for every one of these neurons and subtracting it
细微的图案,但是我们计算了每一个神经元的平均值 然后减去

1:01:55 so whatever bias you add here is going to get subtracted right here and so these biases are
所以不管你在这里添加的偏见 都会在这里被减掉 因此这些偏见

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:02:01 not doing anything in fact they're being subtracted out and they don't impact the rest of the
他们没有做任何事 事实上,他们被减出来 他们不会影响其余的

1:02:06 calculation so if you look at b1.grad it's actually going to be zero because it's being
所以如果看b1.GG 的计算结果,它实际上将是零,因为

1:02:10 subtracted out and doesn't actually have any effect and so whenever you're using batch
减去了,没有实际效果 所以每次你用批量

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:02:14 normalization layers then if you have any weight layers before like a linear or a convo or something
正常的层 如果之前有重层, 比如线性或会议之类的

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:02:19 like that you're better off coming here and just like not using bias so you don't want to use
喜欢,你最好来这里 和只是喜欢不使用偏见 所以你不想使用

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:02:25 bias and then here you don't want to add it because it's that spurious instead we have this
偏见,然后在这里你不想添加它 因为它是荒谬的,而不是我们有

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:02:31 batch normalization bias here and that batch normalization bias is now in charge of the
批次正常化偏重现在负责

batch normalization
批归一化 批归一化
训练与优化

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:02:36 biasing of this distribution instead of this b1 that we had here originally and so basically
而不是我们在这里的 b1,所以基本上

distribution
分布 分布
数学与微积分

1:02:43 the batch normalization layer has its own bias and there's no need to have a bias in the layer
批量正常化层有其自身的偏差, 不需要在层上有偏差

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:02:48 before it because that bias is going to be subtracted out anyway so that's the other small
因为它之前的偏差会被删除 所以这是另一个小

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:02:52 detail to be careful with sometimes it's not going to do anything catastrophic this b1 will
注意细节,有时小心一些... ...它不会做任何灾难性的事情

1:02:57 just be useless it will never get any gradient it will not learn it will stay constant and it's just
没用的,不会得到任何梯度 它不会学到它会保持不变,它只是

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:03:02 wasteful but it doesn't actually really impact anything otherwise okay so i rearranged the
浪费的浪费,但实际上并没有真正影响 任何事情,否则没关系

1:03:07 code a little bit with comments and i just wanted to give a very quick summary of the
代码稍加一点评论, 我只是想对

summary ['sʌməri]
总结 在 TensorFlow 中的某一步 计算出的一个值或一组值，通常用于在训练期间跟踪模型 指标。
神经网络基础

1:03:11 batch normalization layer we are using batch normalization to control the statistics of
批次正常化层 我们正使用批次正常化 来控制

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:03:17 activations in the neural net it is common to sprinkle batch normalization layer across the
神经网中的激活装置通常会将批量正常化层洒到整个神经网的

neural net
神经网络 神经网络
神经网络基础

batch normalization
批归一化 批归一化
训练与优化

1:03:22 neural net and usually we will place it after layers that have multiplications like for example
通常我们会放置在多层的神经网之后, 比如说,

neural net
神经网络 神经网络
神经网络基础

1:03:28 a linear layer or a convolutional layer which we may cover in the future now the batch normalization
一种线性层或一个我们今后可能覆盖的卷变层,现在批量正常化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

convolutional layer
卷积层 深度神经网络的一个层，卷积过滤器 会在其中传递输入矩阵。以下面的 3x3 卷积过滤器 为例： 下面的动画显示了一个由 9 个卷积运算（涉及 5x5 输入矩阵）组成的卷积层。请注意，每个 卷积运算都涉及一个不同的 3x3 输入矩阵切片。由此产生的 3×3 矩阵（右侧）就包含 9 个卷 积运算的结果：
神经网络基础

1:03:34 internally has parameters for the gain and the bias and these are trained using back propagation
内部有收益和偏见的参数,这些参数是利用回传培训的

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

back propagation
反向传播 反向传播
神经网络基础

1:03:41 it also has two buffers the buffers are the mean and the standard deviation the running mean
缓冲是平均值,标准偏差是运行平均值

1:03:47 and the running mean of the standard deviation and these are not trained using back propagation
以及标准差的运行平均值,这些没有用回传来训练

back propagation
反向传播 反向传播
神经网络基础

1:03:52 these are trained using this janky update of kind of like a running mean update so
训练他们使用这个janky更新 有点像一个运行中的平均更新,所以

1:04:01 these are sort of the parameters and the buffers of batch normalization layer
这些都是批量正常化层的参数和缓冲

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:04:05 and then really what it's doing is it's calculating the mean and standard deviation
并且它实际上正在做的是 它在计算平均差和标准差

1:04:08 of the activations that are feeding into the batch normalization over that batch
使批次正常化的集成启动程序

batch normalization
批归一化 批归一化
训练与优化

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:04:13 batch then it's centering that batch to be unit Gaussian and then it's offsetting and scaling it by the
然后它将这批货物集中到高西亚单位 然后它抵消并缩放

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

scaling ['skeiliŋ]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

1:04:19 learned bias and gain and then on top of that it's keeping track of the mean and standard deviation
学习偏见和获得利益,然后又在跟踪中值和标准偏差

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:04:26 of the inputs and it's maintaining this running mean and standard deviation and this will later
输入输入, 它维持着这种运行中平均和标准偏差 以后会这样

1:04:32 be used at inference so that we don't have to re-estimate the mean and standard deviation
用来推断 这样我们不必再估计平均值和标准差

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

1:04:36 all the time and in addition that allows us to basically forward individual examples at test
使我们基本上能够提出个别实例,进行测试

1:04:41 time so that's the batch normalization layer it's a fairly complicated layer
时间 所以这是批量正常化层 这是一个相当复杂的层

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:04:48 but this is what it's doing internally now I wanted to show you a little bit of a real example
但这是内部做 现在我想给你们展示一个真正的例子

1:04:53 so you can search ResNet which is a residual neural network and these are contacts of neural
以便搜索ResNet,这是一个剩余神经网络, 这些是神经的连接

resnet
残差网络 残差网络
神经网络基础

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

1:04:59 networks used for image classification and of course we haven't come to ResNets in detail
用于图像分类的网络, 当然我们还没有详细到ResNets

1:05:04 so I'm not going to explain all the pieces of it but for now just note that
所以我不打算解释所有细节 现在只要指出

1:05:09 the image feeds into a ResNet on the top here and there's many many layers with repeating structure
图像输入到 ResNet 顶部的 RESNet , 并且有很多层与重复结构

resnet
残差网络 残差网络
神经网络基础

1:05:15 all the way to predictions of what's inside that image this repeating structure is made up of these
一直到预测里面的图像

1:05:20 blocks and these blocks are just sequentially stacked up in this deep neural network now the
这些块块只是相继堆叠在这个深神经网络中 现在

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

1:05:26 code for this the block basically that's used and repeated sequentially in series is called
用于此代码的块块, 基本上, 使用并重复序列序列中的区块, 被称为

1:05:32 this bottleneck block bottleneck block and there's a lot here this is all PyTorch and of course we
这儿有很多 这是皮托尔奇 当然我们

pytorch
PyTorch PyTorch
代码与工程

1:05:39 haven't covered all of it but I want to point out some small pieces of it here in the init is where
还没有覆盖所有这一切 但我想指出

1:05:44 we initialize the neural net so this code of block here is basically the kind of stuff we're
我们先开始神经网 所以这里的这个块代码 基本上就是我们所要的东西

neural net
神经网络 神经网络
神经网络基础

1:05:48 doing here we're initializing all the layers and in the forward we are specifying how the
我们在这里做,我们正在开始 所有的层和前方

1:05:53 neural net acts once you actually have the input so this code here is along the lines
一旦您有输入, 神经网络动作就会发生 所以这个代码就是沿着这条线

neural net
神经网络 神经网络
神经网络基础

1:05:57 of what we're doing here and now these blocks are replicated and stacked up serially and that's what
和我们在这里所做的,现在 这些街区被复制 并堆叠成连续的

1:06:06 residual network would be and so notice what's happening here comb1 these are convolution
留置网络将会是,所以注意这里发生的一切 梳子1 这些是卷土重来

convolution [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

1:06:13 layers and these convolution layers basically they're the same thing as a linear layer
和这些卷变层基本上 它们与线性层是一回事

convolution [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:06:19 except convolution layers don't apply convolution layers are used for images and so they have
图像使用 并用

convolution [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

1:06:25 spatial structure and basically this linear multiplication and bias offset are done on patches
空间结构,基本上就是这个线性乘数和偏偏偏偏抵消,是用补丁来进行

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:06:31 instead of a map instead of the full input so because these images have structure spatial
而不是地图,而不是全部输入,所以因为这些图像具有结构空间

1:06:36 structure convolutions just basically do wx plus b but they do it on overlapping patches of the
结构的相叠变化基本上只是 do wx+b, 但是它们对于重叠的

1:06:42 input but otherwise it's wx plus b then we have the normal layer which by default here is
输入,但否则是 wx+b,然后我们有正常层,默认为

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:06:49 initialized to be a batch norm in 2d so two-dimensional batch normalization layer
初始化为2d的批量规范,如此二维批量正常化层

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

norm
范数 范数
数学与微积分

1:06:54 and then we have a non-linearity like relu so instead of here they use relu we are using 10h in this case
然后我们有一个非线性,比如Relu 所以他们在这里使用 Relu 而不是这里 我们在这个情况下使用10小时

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

1:07:02 but both both are just non-linearities and you can just use them at a relatively interchangeably
两者都是非线性,你可以在相对可互换的状态下使用

1:07:07 for very deep networks relus typically empirically work a bit better so see the motif that's being
通常经验上的工作效果更好一些 所以看看现在的动机

1:07:13 repeated here we have convolution batch normalization relu convolution batch normalization
这儿重复了一遍,我们有革命批次 正常化批次 革命批次正常化

convolution [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

batch normalization
批归一化 批归一化
训练与优化

1:07:17 relu etc and then here this is a residual connection that we haven't covered yet
等再在这里 这是我们还没有覆盖的剩余连接

relu
线性修正单元/线性修正函数 Abbreviation for the rectified linear unit, an activation function common in modern neural networks that passes positive inputs unchanged and outputs zero for negative in
数学与微积分

1:07:22 but basically that's the exact same pattern we have here we have a weight layer like a convolution
但基本上,这是相同的模式 我们在这里有同样的模式 我们有一个重层 像卷起

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:07:28 or like a linear layer batch normalization and then 10h which is a non-linearity but basically
或类似线性层的线性分批正常化,然后是10h, 这是一种非线性但基本上

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

batch normalization
批归一化 批归一化
训练与优化

1:07:36 a weight layer a normalization layer and non-linearity and that's the motif that you
重力层 一个正常的层和非线性 这就是你想要的

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:07:40 would be stacking up when you create these deep neural networks exactly as it's done here
当你创造这些深层神经网络时 就会堆叠起来 就像这里所做的那样

1:07:45 and one more thing I'd like you to notice is that here when they are initializing the
还有一件事,我想让你注意 当他们开始

1:07:49 comm layers like comm one by one the depth for that is right here and so it's initializing an nn.com2d
共和层,如一个一个一个一个的通讯层, 深度就在这里,所以它正在初始化 nn.com2d

1:07:56 which is a convolutional layer in PyTorch and there's a bunch of keyword arguments here that I am
皮托尔奇的一座革命层 有很多关键词的争论

convolutional layer
卷积层 深度神经网络的一个层，卷积过滤器 会在其中传递输入矩阵。以下面的 3x3 卷积过滤器 为例： 下面的动画显示了一个由 9 个卷积运算（涉及 5x5 输入矩阵）组成的卷积层。请注意，每个 卷积运算都涉及一个不同的 3x3 输入矩阵切片。由此产生的 3×3 矩阵（右侧）就包含 9 个卷 积运算的结果：
神经网络基础

pytorch
PyTorch PyTorch
代码与工程

1:08:00 not going to explain yet but you see how there's bias equals false the bias equals false is exactly
还没有解释,但你看,偏见如何等于假

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:08:06 for the same reason as bias is not used in our case you see how I raised the use of bias
理由与我们案件没有使用偏见的原因相同,你看我如何提出使用偏见

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:08:12 and the use of bias is spurious because after this weight layer there's a batch normalization
使用偏差是虚假的 因为在这块重量层之后 分批正常

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:08:16 and the batch normalization subtracts that bias and then has its own bias so there's no need to
批次正常化会减去这种偏差,然后有自己的偏差,所以没有必要

batch normalization
批归一化 批归一化
训练与优化

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:08:21 introduce these spurious parameters it wouldn't hurt performance it's just useless and so because
引入这些虚假参数不会伤害性能的 只会没用,所以因为

1:08:26 they have this motif of comm, batch, and rellu they don't need a bias here because there's a bias
他们有这种通、批、批和反折的动机, 他们不需要偏见在这里 因为有偏见

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:08:31 inside here so by the way this example here is very easy to find just do resnet pyTorch
以这里的例子为例,很容易找到 只是些雷纳特·皮托尔奇

resnet
残差网络 残差网络
神经网络基础

pytorch
PyTorch PyTorch
代码与工程

1:08:39 and uh it's this example here so this is kind of like the stock implementation of a residual
举个例子,这里就是这个例子 所以这有点像 剩余存货的库存执行

implementation
实现 实现
代码与工程

1:08:44 neural network in PyTorch and you can find that here but of course I haven't covered many of these
皮托尔奇的神经网络 你在这里可以找到这个 但当然我还没有覆盖过很多这些

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

pytorch
PyTorch PyTorch
代码与工程

1:08:49 parts yet and I would also like to briefly descend into the definitions of these PyTorch layers
我愿简要地谈谈这些俾火石层的定义。

pytorch
PyTorch PyTorch
代码与工程

1:08:55 and the parameters that they take now instead of a convolutional layer we're going to look at
以及他们现在的参数 而不是我们将要研究的 革命层

convolutional layer
卷积层 深度神经网络的一个层，卷积过滤器 会在其中传递输入矩阵。以下面的 3x3 卷积过滤器 为例： 下面的动画显示了一个由 9 个卷积运算（涉及 5x5 输入矩阵）组成的卷积层。请注意，每个 卷积运算都涉及一个不同的 3x3 输入矩阵切片。由此产生的 3×3 矩阵（右侧）就包含 9 个卷 积运算的结果：
神经网络基础

1:08:59 a linear layer because that's the one that we're using here this is a linear layer and I
线性层,因为我们在这里使用的是这个线性层。 这是一个线性层和我

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:09:04 haven't covered covered convolutions yet but as I mentioned convolutions are basically
还没有覆盖过革命的卷起,但正如我提到的, 基本上就是

1:09:07 linear layers except on patches so a linear layer performs a wx plus b except here they're
线性层除补丁外,均线性层的 wx+b 显示除此处外

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:09:15 calling the w a transpose um so it's called Wx plus b very much like we did here to initialize this
叫做Wx+B 就像我们在这里做的一样

1:09:22 layer you need to know the fan in the fan out and that's so that they can initialize this w this
需要了解扇扇扇的风扇,这样他们就可以初始化这个 ww

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:09:29 is the fan in and the fan out so they know how how big the weight matrix should be you need to
扇扇门在和扇扇外,让他们知道你需要多少重量矩阵

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

matrix
矩阵 矩阵
数学与微积分

1:09:35 also pass in whether you whether or not you want a bias and if you set it to false then no bias will be
是否想要偏见,如果以偏见为假,那么,无论你是否愿意,无论你是否有偏见,无论你把它设为假的,无论你用不存偏见,都不要有偏见。

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:09:41 inside this layer um and you may want to do that exactly like in our case if your layer is followed
和我们的情况一样 如果你的层被跟着

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:09:48 by a normalization layer such as batch norm so this allows you to basically disable bias in terms
由正常的层(如批量规范)来设置,这样可以基本上消除偏差。

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:09:54 of the initialization if we swing down here this is reporting the variables used inside this
初始化的初始化, 如果我们在此向下摇动, 这是报告此内使用的变量

1:09:59 linear layer and our linear layer here has two parameters the weight and the bias in the same
线性图层和我们的线性图层次在此有两参数,即重量和偏向,以相同

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:10:06 way they have a weight and a bias and they're talking about how they initialize it by default
他们的体重和偏重 都在谈论他们如何默认初始化

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

动量 · 批次大小 1:10:11

1:10:11 so by default pythorch will initialize your weights by taking the fan in and then um doing one over
所以默认的 pythorch 将会通过使用扇子开始初始化您的重量, 然后用 um 来做一个

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:10:18 fan in square root and then instead of a normal distribution they are using a uniform
平方根中扇扇形,然后用制服代替正常分布

distribution
分布 分布
数学与微积分

1:10:24 distribution so it's very much the same thing but they are using a one instead of five over three so
分布分布,所以它非常一样 但是他们用的是一台而不是五乘三

distribution
分布 分布
数学与微积分

1:10:30 there's no gain being calculated here the gain is just one but otherwise is exactly one over the square
这里没有计算收益 收益只是一个,否则就是方形上的

1:10:36 root of fan in exactly as we have here so one over the square root of k is the is the scale of the
根的扇子,和我们在这里一样, 所以K平方根上的是

1:10:44 weights but when they are drawing the numbers they're not using a Gaussian by default they're
数数的计算时,它们没有使用高斯语默认值

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:10:48 using a uniform distribution by default and so they draw uniformly from negative square root
使用默认情况下统一分布, 从而统一从负平方根中提取

distribution
分布 分布
数学与微积分

1:10:53 of k to square root of k but it's the exact same thing and the same motivation from before with respect
k 到 k 平方根的 k, 但这是完全相同的事物 和之前同样的动机 在尊重

1:11:00 to what we've seen in this lecture and the reason they're doing this is if you have a roughly Gaussian
和我们在这次演讲中所看到的一样 他们之所以这么做 是因为如果你有一个大概高山

1:11:05 input this will ensure that out of this layer you will have a roughly Gaussian output and use
这将确保您在这个层中 大致有高斯输出和使用

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:11:12 you basically achieve that by scaling the weights by one over the square root of fan in
您基本上实现了这一点,通过将重量乘以一个在

scaling ['skeiliŋ]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:11:17 so that's what this is doing and then the second thing is the basher normalization layer
所以这就是我们要做的,然后第二件事是 冲撞器的正常化层

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:11:23 so let's look at what that looks like in pytorch so here we have a one-dimensional
让我们看看在火炬里是什么样子 所以这里我们有一个单维

pytorch
PyTorch PyTorch
代码与工程

1:11:27 basher normalization layer exactly as we are using here and there are a number of keyword
与我们在此使用的正本平面正一样, 有一些关键字

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:11:31 arguments going into it as well so we need to know the number of features for us that is 200
也有人在讨论其中的论据, 所以我们需要知道我们有多少特征,

1:11:37 and that is needed so that we can initialize these parameters here the gain the bias and the
并且我们需要这个需要,以便我们可以在这里开始这些参数 获得偏见和

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:11:42 buffers for the running mean and standard deviation then they need to know the value
运行平均值和标准偏差的缓冲符和标准偏移,然后它们需要知道值

1:11:48 of epsilon here and by default this is one negative five you don't typically change this too much
默认这里的欧西隆, 默认情况下, 这是1负5, 您通常不会改变太多

1:11:53 then they need to know the momentum and the momentum here as they explain is basically used
然后它们需要了解这里的动力和势头,因为它们解释是基本使用的

momentum [məu'mentəm]
动量 (https://www.tensorflow.org/api_docs/python/tf/train/MomentumOptimizer) 更新频率（AdaGrad (https://www.tensorflow.org/api_docs/python/tf/train/AdagradOptimizer) = ADAptive GRADient descent； Adam (https://www.tensorflow.org/api_docs/python/tf/train
训练与优化

1:11:59 for these uh running mean and running standard deviation so by default the momentum here is
运行中的平均和标准偏差 所以默认这里的势头是

momentum [məu'mentəm]
动量 (https://www.tensorflow.org/api_docs/python/tf/train/MomentumOptimizer) 更新频率（AdaGrad (https://www.tensorflow.org/api_docs/python/tf/train/AdagradOptimizer) = ADAptive GRADient descent； Adam (https://www.tensorflow.org/api_docs/python/tf/train
训练与优化

1:12:04 point one the momentum we are using here in this example is 0.001 and basically rough
指出我们在此例中使用的 势头之一,是0.001,基本上粗糙

momentum [məu'mentəm]
动量 (https://www.tensorflow.org/api_docs/python/tf/train/MomentumOptimizer) 更新频率（AdaGrad (https://www.tensorflow.org/api_docs/python/tf/train/AdagradOptimizer) = ADAptive GRADient descent； Adam (https://www.tensorflow.org/api_docs/python/tf/train
训练与优化

1:12:11 you may want to change this sometimes and roughly speaking if you have a very large batch size
如果您的批量很大, 可能会想改变这个过程。

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:12:17 then typically what you'll see is that when you estimate the mean and standard deviation
通常你会看到 当你估计平均差和标准差时

1:12:21 for every single batch size if it's large enough you're going to get roughly the same result
每一批量大小,如果足够大, 你会得到大致相同的结果

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:12:26 and so therefore you can use slightly higher momentum like 0.1 but for a batch size as small as 32
因此,你可以使用略高的动力力,像0.1一样,但对于小到32个批量体积的小部分,

momentum [məu'mentəm]
动量 (https://www.tensorflow.org/api_docs/python/tf/train/MomentumOptimizer) 更新频率（AdaGrad (https://www.tensorflow.org/api_docs/python/tf/train/AdagradOptimizer) = ADAptive GRADient descent； Adam (https://www.tensorflow.org/api_docs/python/tf/train
训练与优化

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:12:34 the mean understanding deviation here might take on slightly different numbers because there's only
表示理解偏差的平均值 这里可能取用略微不同的数字 因为只有

1:12:38 32 examples we are using to estimate the mean and standard deviation so the value is changing
我们使用的32个例子用来估计平均值和标准差,因此价值正在变化

1:12:42 around a lot and if your momentum is 0.1 that that might not be good enough for this value to settle
大约很多,如果你的势头是0.1, 那可能不够好 这个价值稳定下来

momentum [məu'mentəm]
动量 (https://www.tensorflow.org/api_docs/python/tf/train/MomentumOptimizer) 更新频率（AdaGrad (https://www.tensorflow.org/api_docs/python/tf/train/AdagradOptimizer) = ADAptive GRADient descent； Adam (https://www.tensorflow.org/api_docs/python/tf/train
训练与优化

1:12:49 and converge to the actual mean and standard deviation over the entire training set and so
并会合到整个培训组的实际平均和标准差值,等等

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:12:55 basically if your batch size is very small momentum of 0.1 is potentially dangerous and it might make

基本上如果你的批量大小很小的话 0.1的动力力非常小 可能很危险

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

momentum [məu'mentəm]
动量 (https://www.tensorflow.org/api_docs/python/tf/train/MomentumOptimizer) 更新频率（AdaGrad (https://www.tensorflow.org/api_docs/python/tf/train/AdagradOptimizer) = ADAptive GRADient descent； Adam (https://www.tensorflow.org/api_docs/python/tf/train
训练与优化

1:13:00 it so that the running mean and standard deviation is thrashing too much during training

使得运行中的平均和标准偏差在培训期间受到太多的打击

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:13:05 and it's not actually converging properly affine equals true determines whether this

并且它实际上没有相融合 等于确定这个是否

1:13:11 batch normalization layer has these learnable affine parameters the the gain and the bias

批次正常化层有这些可学习的同系物参数,以收益和偏差为基准

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:13:18 and this is almost always kept to true I'm not actually sure why you would want to change this to

几乎总是保持真实的 我不知道为什么你要改变这个

1:13:23 false then track running stats is determining whether or not batch normalization layer of
运行时的状态数据正在确定

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:13:30 pytorch will be doing this and one reason you may you may want to skip the running stats
烧火机会这样做,一个原因 你可以跳过运行中的统计

pytorch
PyTorch PyTorch
代码与工程

1:13:37 is because you may want to for example estimate them at the end as a stage two
是因为您可能想要在结尾处作为第二阶段来估计这些数值。

1:13:42 like this and in that case you don't want the batch normalization layer to be doing all this
这样,如果这样的话 你不想让批量正常化层做这些

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:13:45 extra compute that you're not going to use and finally we need to know which device we're
计算你不会用什么 最终我们需要知道我们是哪个装置

device [di'vais]
设备 一类可运行 TensorFlow 会话的硬件，包括 CPU 、 GPU 和 TPU 。
神经网络基础

1:13:51 going to run this batch normalization on a cpu or a gpu and what the data type should be
运行此批量在 cpu 或 gpu 上正常化, 以及数据类型应该是什么

batch normalization
批归一化 批归一化
训练与优化

1:13:56 half precision single precision double precision and so on so that's the batch normalization
精准度半精准度 精确度 双精度 双精确度等等 这样分批正常

precision [pri'siʒən]
精确率 一种分类模型 指标。精确率指模型正确预测正类别 的频 率，即:
神经网络基础

batch normalization
批归一化 批归一化
训练与优化

1:14:02 layer otherwise they link to the paper is the same formula we've implemented and everything is
否则他们和纸的链接 是我们所执行的公式 一切

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

随机梯度下降 · PyTorch 1:14:07

1:14:07 the same exactly as we've done here okay so that's everything that I wanted to cover for this lecture
和我们在这里所做的一样,好吗? 好,这就是我想为这个演讲 报道的一切

1:14:13 really what I wanted to talk about is the importance of understanding the activations
我想说的是 理解启动的重要性

1:14:17 and the gradients and their statistics in neural networks and this becomes increasingly important
以及神经网络中的梯度及其统计数据,这变得越来越重要

1:14:21 especially as you make your neural networks bigger larger and deeper we looked at the
尤其是当你让神经网络 变得更大更深入 我们看着

1:14:26 distributions basically at the output layer and we saw that if you have two confident
基本上在输出层的分布 我们发现如果两个有自信

output layer
输出层 神经网络的 " 最后 " 一层，也是包含答案的层。
神经网络基础

1:14:30 mis predictions because the activations are too messed up at the last layer you can end up
错误预测 因为激活在最后一层太乱了 你最终会变成

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:14:35 with these hockey stick losses and if you fix this you get a better loss at the end of training
与这些曲棍球棒损失 如果你解决这个问题, 你会得到更好的损失 在训练结束时

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:14:40 because your training is not doing wasteful work then we also saw that we need to control the
因为你们的训练没有浪费工作, 我们也看到我们需要控制

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:14:45 activations we don't want them to you know squash to zero or explode to infinity and because that
我们不希望他们知道 壁球到零或爆炸到无限

1:14:51 you can run into a lot of trouble with all of these non-linearities in these neural nets
神经网中这些非线性的东西 可能会遇到很多麻烦

1:14:55 and basically you want everything to be fairly homogeneous throughout the neural net you
基本上你希望所有的东西 都相当单一 在神经网里,你

neural net
神经网络 神经网络
神经网络基础

1:14:58 want roughly Gaussian activations throughout the neural net then we talked about okay if we
大约需要高斯启动 整个神经网的神经网络 然后我们讨论好了 如果我们

neural net
神经网络 神经网络
神经网络基础

1:15:04 would roughly Gaussian activations how do we scale these weight matrices and biases during
大约高斯会激活高斯启动 我们如何在

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:15:09 initialization of the neural net so that we don't get you know so everything is as controlled as possible
神经网络初始化 这样我们就无法让你知道 所以一切都尽可能控制

neural net
神经网络 神经网络
神经网络基础

1:15:17 so that gave us a large boost in improvement and then I talked about how that strategy is not
从而大大推动我们改进, 然后我谈到这个战略是如何不

1:15:22 actually possible for much much deeper neural nets because when you have much deeper neural
更深得多的神经网 实际上可以 因为当你有更深得多神经网

1:15:29 nets with lots of different types of layers it becomes really really hard to precisely set
具有许多不同类型层的网状图, 它变得真的很难精确设置

1:15:35 the weights and the biases in such a way that the activations are roughly uniform throughout the neural
其重量和偏差 使得激活在神经中大致一致

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:15:40 net so then I introduced the notion of a normalization layer now there are many normalization
开始使用网络,然后我提出一个正常化层的概念。 现在有许多正常化

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:15:45 layers that that people use in practice batch normalization layer normalization
分批正常化层的正常化

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:15:50 instant normalization group normalization we haven't covered most of them but I've introduced
即时正常化小组的正常化 我们还没有覆盖他们的大部分 但我已经介绍

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:15:54 the first one and also the one that I believe came out first and that's called batch normalization
第一个也是我认为第一个 被我先推出的,叫做批量正常化

batch normalization
批归一化 批归一化
训练与优化

1:16:00 and we saw how batch normalization works this is a layer that you can sprinkle throughout your
我们看到批量正常化是如何运作的 这是一层你可以在你的

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:16:05 deep neural net and the basic idea is if you want roughly Gaussian activations well then take your
基本的想法是 如果你想让高山的活化功能顺利进行 那么就把你的

neural net
神经网络 神经网络
神经网络基础

1:16:11 activations and take the mean understanding deviation and center your data and you can do
激活并使用中值理解偏移, 并集中数据, 您可以做

1:16:17 that because the centering operation is differentiable but and on top of that we actually had to
因为中心行动是不同的 但是,最重要的是 我们实际上必须

operation
运算 运算
神经网络基础

1:16:23 add a lot of bells and whistles and that gave you a sense of the complexities of the
加上许多钟声和哨声,这让你感觉到了

1:16:27 batch normalization layer because now we're centering the data that's great but suddenly we
批量正常化层 因为现在我们正集中数据

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:16:31 need the gain and the bias and now those are trainable and then because we are coupling all the
需要利益和偏见 而现在这些是训练的 然后因为我们把所有

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:16:37 training examples now suddenly the question is how do you do the inference where to do the
训练的例子突然间就突然问到: 你如何推断出在哪里可以做到

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

1:16:42 inference we need to now estimate these mean and center deviation once over the entire
假设我们现在需要估计一下这些中位和中位偏差

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

1:16:49 training set and then use those at inference but then no one likes to do stage two so instead
训练组装,然后在推断时使用这些训练,但没有人喜欢做第二阶段,所以改为

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

1:16:54 we fold everything into the batch normalization layer during training and try to estimate these
我们把所有东西都折叠在 训练过程中的批量正常化层中 并试着估算这些

batch normalization
批归一化 批归一化
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:16:59 in a running manner so that everything is a bit simpler and that gives us the batch normalization
以运行的方式,这样一切都会简单一点 这样我们就能实现批量正常化

batch normalization
批归一化 批归一化
训练与优化

1:17:04 layer and as I mentioned no one likes this layer it causes a huge amount of bugs and intuitively
正如我提到的,没有人喜欢这个层 引起大量的虫子和直觉

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:17:13 it's because it is coupling examples in the forward pass of the neural net and I've shot
因为它把神经网前传的范例 合并在一起 我拍了

forward pass
前向传播 前向传播
神经网络基础

neural net
神经网络 神经网络
神经网络基础

1:17:20 myself in the foot with this layer over and over again in my life and I don't want you to suffer
脚踏在脚下,我人生中这层层 一遍又一遍地重复 我不想你受苦

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:17:26 the same so basically try to avoid it as much as possible some of the other alternatives to these
基本上尽量避免它,尽可能避免使用其他替代方法。

1:17:33 layers are for example group normalization or layer normalization and those have become more
例如,组的正常化或层的正常化,这些已经变得更

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:17:37 common in more recent deep learning but we haven't covered those yet but definitely
近来深层学习中常见的 但是我们还没有研究过这些 但肯定

1:17:43 batch normalization was very influential at the time when it came out in roughly 2015
批量正常化在2015年左右推出时,具有很大影响力

batch normalization
批归一化 批归一化
训练与优化

1:17:48 because it was kind of the first time that you could train reliably much deeper neural nuts
因为这是第一次 你可以训练更深的神经坚果

1:17:55 and fundamentally the reason for that is because this layer was very effective at controlling
其根本原因在于这个层在控制

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:17:59 the statistics of the activations in a neural net so that's the story so far and that's all I
神经网中激活的统计数据 所以这是目前为止的故事

neural net
神经网络 神经网络
神经网络基础

1:18:06 wanted to cover and in the future lecture hopefully we can start going into recurrent
希望我们能开始重复

1:18:10 neural nets and recurrent neural nets as we'll see are just very very deep networks
神经网和经常神经网 我们可以看到,这些网络非常深

1:18:16 because you you unroll the loop and when you actually optimize these neural nets and that's
因为你打开了环圈 当你真正优化这些神经网时

1:18:21 where a lot of this analysis around the activation statistics and all these normalization layers
围绕激活统计数据和这些正常化层 进行了很多分析

activation
激活值 激活值
神经网络基础

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:18:29 will become very very important for good performance so we'll see that next time bye
表现良好,就变得非常重要了 下次再见

1:18:35 okay so I lied I would like us to do one more summary here as a bonus and I think it's useful
所以我撒谎了 我想我们再做一个总结 作为奖金 我认为这很有用

summary ['sʌməri]
总结 在 TensorFlow 中的某一步 计算出的一个值或一组值，通常用于在训练期间跟踪模型 指标。
神经网络基础

1:18:40 as to have one more summary of everything I've presented in this lecture but also I would
还要再总结一下我在这篇演讲中介绍的一切 但我也愿意

summary ['sʌməri]
总结 在 TensorFlow 中的某一步 计算出的一个值或一组值，通常用于在训练期间跟踪模型 指标。
神经网络基础

1:18:44 like us to start by torturing our code a little bit so it looks much more like what you would encounter
开始先折磨我们的代码一点点 这样看起来更像是你会遇到的

1:18:48 in pytorch so you'll see that I will structure our code into these modules like a linear module
以火炬的形式,你可以看到 我会把代码结构成这些模块, 像线性模块一样

pytorch
PyTorch PyTorch
代码与工程

1:18:56 and a batch storm module and I'm putting the code inside these modules so that we can construct
和批量风暴模块 我将代码放入这些模块中 这样我们就可以构建

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:19:02 neural networks very much like we would construct them in pytorch and I will go through this in
和我们建造的热电网一样 我将在

pytorch
PyTorch PyTorch
代码与工程

1:19:05 detail so we'll create our neural net then we will do the optimization loop as we did before
细节 所以我们会创建神经网 然后我们像以前一样做优化循环

neural net
神经网络 神经网络
神经网络基础

1:19:12 and then the one more thing that I want to do here is I want to look at the activation statistics
我想在这里做的是 我想看看激活统计

activation
激活值 激活值
神经网络基础

1:19:16 both in the forward pass and in the backward pass and then here we have the evaluation and
无论是在前门和后门,然后在这里我们得到了评估

forward pass
前向传播 前向传播
神经网络基础

backward pass
反向传播 反向传播
神经网络基础

1:19:20 sampling just like before so let me rewind all the way up here and go a little bit slower
和以前一样,采样就像以前一样 所以让我往上倒一圈 再慢一点

1:19:26 so here I am creating a linear layer you'll notice that torch dot and n has lots of different
所以我在这里要建立一个线性层, 你会注意到点和点的火炬有很多不同

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:19:31 types of layers and one of those layers is the linear layer torch dot and n dot linear takes
线性层点火炬和 n点线性取用

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:19:36 a number of input features output features whether or not we should have bias and then the device
是否应该有偏差, 然后是设备

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

device [di'vais]
设备 一类可运行 TensorFlow 会话的硬件，包括 CPU 、 GPU 和 TPU 。
神经网络基础

1:19:40 that we want to place this layer on and the data type so I will omit these two but otherwise we have
想要把这个层和数据类型放上, 所以我会略去这两个部分, 否则我们有

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:19:46 the exact same thing we have the fan in which is the number of inputs fan out the number of outputs
输入数量 也从输出的数量中发出

1:19:53 and whether or not we want to use a bias and internally inside this layer there's a weight
以及我们是否想要使用偏见 内部在这个层内,有重量

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:19:57 and a bias if you like it it is typical to initialize the weight using say random numbers
偏差,如果喜欢的话, 使用随机数字来初始化重量是典型的

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:20:04 drawn from agassian and then here's the coming initialization that we discussed already in this
由气相学引出,然后我们讨论的即将开始的

1:20:09 lecture and that's a good default and also the default that I believe pytorch uses and by default
并且这是一个良好的违约, 以及默认的默认 我相信烧火机的使用和默认

pytorch
PyTorch PyTorch
代码与工程

1:20:15 the bias is usually initialized to zeros now when you call this module this will basically
当您调用这个模块时, 偏向通常初始化为零, 现在它会基本为零

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

1:20:21 calculate w times x plus b if you have nb and then when you also call that parameters on this
计算 w乘 x x+ b ,如果有 nb ,然后您同时调用此参数

1:20:26 module it will return the tensors that are the parameters of this layer now next we have the
模块中返回的 Exaron 将返回该层参数中的 数组。

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:20:33 bash normalization layer so I've written that here and this is very similar to pytorch 's nn dot
平面平整层,所以我写了这个 这里和Pytorch的NN点非常相似

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:20:40 bash norm 1d layer as shown here so I'm kind of taking these three parameters here the
dimensionality
标准 1d 层, 这里显示的 标准 1D 层, 所以我要在这里使用这三个参数

norm
范数 范数
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:20:49 the epsilon that we'll use in the division and the momentum that we will use in keeping track
以追踪我们所使用的势头

momentum [məu'mentəm]
动量 (https://www.tensorflow.org/api_docs/python/tf/train/MomentumOptimizer) 更新频率（AdaGrad (https://www.tensorflow.org/api_docs/python/tf/train/AdagradOptimizer) = ADAptive GRADient descent； Adam (https://www.tensorflow.org/api_docs/python/tf/train
训练与优化

1:20:53 of these running stats the running mean and the running variance now pytorch actually takes
运行中的燃烧火实际需要的

pytorch
PyTorch PyTorch
代码与工程

1:20:59 quite a few more things but I'm assuming some of their settings so for us alpha will be true
还有很多其他的事情 但我假设他们的一些设置 所以对我们来说,阿尔法是真实的

1:21:03 that means that we will be using a gamma and beta after the normalization the track running stats
这意味着在正常运行后,我们将使用伽玛和贝贝 轨道运行状态

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:21:08 will be true so we will be keeping track of the running mean and the running variance in the in
将会是真实的,所以我们将追踪运行中的平均值和

1:21:13 the bash norm our device by default is the cpu and the data type by default is float 32
击球规范,默认情况下我们设备是 cpu ,默认情况下数据类型是 floc 32

norm
范数 范数
数学与微积分

device [di'vais]
设备 一类可运行 TensorFlow 会话的硬件，包括 CPU 、 GPU 和 TPU 。
神经网络基础

1:21:22 so those are the defaults otherwise we are taking all the same parameters in this bash norm
这些都是默认的,否则我们在这个标准中 将采用所有相同的参数

norm
范数 范数
数学与微积分

1:21:26 layer so first I'm just saving them now here's something new there's dot training which by default
下层层 所以首先我正在保存它们 现在这里有新的东西 有点训练

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:21:32 is true and pytorch nn modules also have this attribute dot training and that's because many
和 Pytonch nn 模块也包含此属性点训练, 因为很多

pytorch
PyTorch PyTorch
代码与工程

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:21:38 modules and bash norm is included in that have a different behavior whether you are training your
模块和标准击击, 包括了以下模块中的行为方式不同, 即您是否正在训练

norm
范数 范数
数学与微积分

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:21:44 normal or whether you are running it in an evaluation mode and calculating your evaluation
或是否在一种评价模式中运行并计算您的评价

1:21:49 laws or using it for inference on some test examples and bash norm is an example of this
法律或用法律推断一些测试实例和标准打击行为就是这方面的一个例子。

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

norm
范数 范数
数学与微积分

1:21:54 because when we are training we are going to be using the mean and the variance estimated from the
因为我们在训练时,将使用平均值和估计的与

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:21:58 current batch but during inference we are using the running mean and running variance and so also if we
目前的批次,但在推断期间 我们使用运行中平均值 并产生差异

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

1:22:05 are training we are updating mean and variance but if we are testing then these are not being
训练是训练,我们正在更新中度和差异,但如果我们测试的话,这些不是

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:22:09 updated they're kept fixed and so this flag is necessary and by default true just like in
更新后它们被固定,所以这面旗帜是必要的 默认是真实的 就像在

1:22:14 pytorch now the parameters of bash norm 1d are the gamma and the beta here and then the running

弹弓现在的弹射标准1D参数 是伽马和贝塔 这里然后运行

pytorch
PyTorch PyTorch
代码与工程

norm
范数 范数
数学与微积分

1:22:22 mean and running variance are called buffers in pytorch nomenclature and these buffers are trained

平板命名法中称为缓冲剂,这些缓冲剂经过培训

pytorch
PyTorch PyTorch
代码与工程

1:22:30 using exponential moving average here explicitly and they are not part of the back propagation

使用指数移动平均数在此明确使用,它们不属于后传播的一部分

exponential
指数 指数
数学与微积分

back propagation
反向传播 反向传播
神经网络基础

1:22:35 stochastic gradient descent so they are not sort of like parameters of this layer and that's why

随机梯度梯度下降 所以它们不象这个层的参数 这就是为什么

stochastic gradient descent
随机梯度下降 随机梯度下降
反向传播与梯度

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:22:40 when we calculate when we have a parameters here we only return gamma and beta we do not return

当我们计算参数时, 当我们在这里有一个参数时, 我们只返回伽马和贝塔, 我们不返回

1:22:45 the mean and the variance this is trained sort of like internally here every forward pass using
平均和差值是训练过的, 就像这里的每一个前方通道使用

forward pass
前向传播 前向传播
神经网络基础

1:22:51 exponential moving average so that's the initialization now in a forward pass if we are training
指数移动平均速度,所以如果我们正在训练, 这就是在前传中初始化

exponential
指数 指数
数学与微积分

forward pass
前向传播 前向传播
神经网络基础

1:22:59 then we use the mean and the variance estimated by the batch let me pull up the paper here we
然后我们用平均值和 批量估计的差异来计算

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:23:06 calculate the mean and the variance now up above i was estimating the standard deviation and keeping
计算平均值和目前高于我以上差异的数值,

1:23:12 track of the standard deviation here in the running standard deviation instead of running
运行标准差中的标准偏差轨迹,而不是运行

1:23:17 variance but let's follow the paper exactly here they calculate the variance which is the
差异值是

1:23:22 standard deviation squared and that's what's kept track of in the running variance instead of a running
标准差平方,这就是在变化中跟踪的变量而不是运行

1:23:27 standard deviation but those two would be very very similar i believe if we are not training
标准偏离,但我认为如果我们不训练,这两个标准偏离非常相似

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:23:34 then we use the running mean and variance we normalize and then here i'm calculating the
然后我们用运行的平均值和差异 来进行正常化,然后在这里计算

1:23:40 output of this layer and i'm also assigning it to an attribute called dot out now dot out is
此层输出的输出, 我也将将其指派给一个属性, 名为点, 现在点出来

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

Adam 优化器 · 链式法则 1:23:46

1:23:46 something that i'm using in our modules here this is not what you would find in pi torch we are
这东西我用在我们的模块中... ...这不是你能找到的,我们是

1:23:51 slightly deviating from it i'm creating a dot out because i would like to very easily maintain all
稍微偏离了一下,我正在制作一个点出来 因为我想很容易地维持所有

1:23:57 those variables so that we can create statistics of them and plot them but a pi torch and modules
这些变量,以便我们可以建立它们的统计数据, 并绘制它们,但一个火炬和模块

1:24:02 will not have a dot out attribute and finally here we are updating the buffers using again as i
将没有点外属性, 最后我们在这里更新缓冲器, 重新使用 i

1:24:08 mentioned exponential moving average uh provide given the provided momentum and importantly you'll
提供指数移动平均 uh 提供 以提供的势头

exponential
指数 指数
数学与微积分

momentum [məu'mentəm]
动量 (https://www.tensorflow.org/api_docs/python/tf/train/MomentumOptimizer) 更新频率（AdaGrad (https://www.tensorflow.org/api_docs/python/tf/train/AdagradOptimizer) = ADAptive GRADient descent； Adam (https://www.tensorflow.org/api_docs/python/tf/train
训练与优化

1:24:13 notice that i'm using the torsion no grad context manager and i'm doing this because if we don't use
注意我正在使用 抗冲没有研究生背景经理的 并且我这样做是因为如果我们不使用

1:24:19 this then pi torch will start building out an entire computational graph out of these
开始从这些中建立整个计算图

1:24:23 tensors because it is expecting that we will eventually call it dot backward but we are
因为它期望我们最终会称它为"点向后" 但我们却在

1:24:28 never going to be calling dot backward on anything that includes running mean and
永远也不会叫点落后 在任何包括运行的卑鄙和

1:24:31 running variance so that's why we need to use this context manager so that we are not
变化变化,所以我们才需要使用这个上下文管理员,这样我们才能不

1:24:36 sort of maintaining them using all this additional memory so this will make it more
使用所有这些额外的内存来保持它们, 这样它就会增加

1:24:41 efficient and it's just telling pi torch that they're really no backward we just have a bunch of tensors
效率高,只是告诉皮火炬 他们真的没有落后 我们只有一堆的

1:24:45 we want to update them that's it and then we return okay now scrolling down we have the 10h layer
我们要更新它们,就这样 然后我们返回好 现在滚下来 我们有10层

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:24:52 this is very very similar to torsion dot 10h and it doesn't do too much it just calculates 10h as
这非常相似的 与10度10度的血压非常相似 而且它不会做太多, 它只是计算了10度

1:24:59 you might expect so that's torsion dot 10h and there's no parameters in this layer but because
也就是10度的腐蚀度, 这个层里没有参数, 因为

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:25:05 these are layers it now becomes very easy to sort of like stack them up into basically just a list
基本上只是一份清单

1:25:13 and we can do all the initializations that we're used to so we have the initial sort of embedding
我们可以做所有我们习惯的初始化, 所以我们有最初的嵌入类型

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:25:18 matrix we have our layers and we can call them sequentially and then again with torsion no grad
分层,我们有我们的层次 我们可以依顺序叫它们 然后再用不等级的腐蚀来再次叫它们

matrix
矩阵 矩阵
数学与微积分

1:25:24 there's some initializations here so we want to make the output softmax a bit less confident
这里有一些初始化的功能 所以我们想降低输出软体的自信

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:25:29 like we saw and in addition to that because we are using a six layer multi- layer perceptron
因为我们使用的是六层多层的多层概念

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:25:34 here so you see how i'm stacking linear 10h linear 10h etc i'm going to be using the game
这儿,你看我如何堆叠线性10小时线性10秒等等 我要用这个游戏

1:25:40 here and i'm going to play with this in a second so you'll see how when we change this what happens
我马上就要玩这个了 这样你就会看到 当我们改变这一切时

1:25:45 to the statistics finally the parameters are basically the embedding matrix and all the
参数基本上就是嵌入矩阵和所有

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

matrix
矩阵 矩阵
数学与微积分

1:25:50 parameters in all the layers and notice here i'm using a double list comprehension if you
在所有层次和注意的参数中的所有参数的分层和注意,

1:25:55 want to call it that but for every layer in layers and for every parameter in each of those layers
想要这样称呼它,但对于每一层层层和每一层的参数,

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

parameter [pə'ræmitə]
参数 机器学习系统自行训练的模型的变量。例如，权重 就是一种参数，它们的值是机器 学习系统通过连续的训练迭代逐渐学习到的。与超参数 相对。
神经网络基础

1:26:00 we are just stacking up all those piece all those parameters now in total we have 46 000 parameters
我们只是堆叠了所有的这些碎片 现在所有参数

1:26:09 and i'm telling pytorch that all of them require gradient then here we have everything here we
我告诉皮条客,他们都需要梯度 然后在这里我们拥有一切

pytorch
PyTorch PyTorch
代码与工程

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:26:18 are actually mostly used to we are sampling batch we are doing forward pass the forward
大部分是用来做抽样的 我们往前做的样本批次 向前走

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

forward pass
前向传播 前向传播
神经网络基础

1:26:23 pass now is just the linear application of all the layers in order followed by the cross entropy
现在的过道只是所有层的线性应用, 顺序依次为十字方的 entropy

entropy
熵 熵
数学与微积分

1:26:29 and then in the backward pass you'll notice that for every single layer i now iterate over
然后在后过后过道中 你会注意到,对于每一个层

backward pass
反向传播 反向传播
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:26:33 all the outputs and i'm telling pytorch to retain the gradient of them and then here we are already
所有的输出结果 我告诉皮火烧厂 保留它们的梯度 然后我们在这里

pytorch
PyTorch PyTorch
代码与工程

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:26:38 used to all the all the gradients set to none do the backward to fill in the gradients do an update
用于所有设置为无的梯度的所有梯度时, 所有常用的梯度都使用, 后向填入渐变时, 更新

1:26:44 using stochastic gradient send and then track some statistics and then i am going to break
使用随机梯度发送, 然后跟踪一些统计数据, 然后我会打破

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:26:50 after a single iteration now here in this cell in this diagram i'm visualizing the histogram
之后,在这张图中的这个单元格中,一个迭代后, 我将直方图直观化

iteration [.itə'reiʃən]
迭代 模型的权重在训练期间的一次更新。迭代包含计算参数在单批次 数据上的梯度损失。 K k-means 一种热门的聚类 算法，用于对非监督式学习中的样本进行分组。 k-means 算法基 本上会执行以下操作： 以迭代方式确定最佳的 k 中心点（称为形心 ）。 将每个样本分配到最近的形心。与同一个形心距离最近的样本属于同一个组。 k-means
神经网络基础

1:26:55 the histograms of the forward pass activations and i'm specifically doing it at the 10h layers
前进传承激活的直方图 我特别在10个H层做

forward pass
前向传播 前向传播
神经网络基础

1:27:01 so iterating over all the layers except for the very last one which is basically just the
在所有层层上循环 除了最后一层

1:27:07 softmax layer if it is a 10h layer and i'm using a 10h layer just because they have a finite output
如果是10H层,我使用10H层的软负层, 仅仅因为输出有限

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:27:14 negative one to one and so it's very easy to visualize here so you see negative one to one
负对1,所以这里很容易想象 所以看一对一

1:27:18 and it's a finite range and easy to work with i take the out tensor from that layer into t
并且它是一个有限的范围,很容易工作 我把从那层的抗拉带出来

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:27:25 and then i'm calculating the mean the standard deviation and the percent saturation of t
然后我计算出标准差和 吨的饱和率

1:27:30 and the way i define the percent saturation is that t.absolute value is greater than 0.97
并且我定义百分率饱和度的方法是 t. absolute 值大于 0.97

1:27:35 so that means we are here at the tails of the 10h and remember that when we are in the tails of the
也就是说我们在这里 在10小时的尾巴 并记住当我们在

1:27:40 10h that will actually stop gradients so we don't want this to be too high now here i'm calling
10h,这实际上会停止梯度 所以我们不希望这个太高 现在这里我打电话

1:27:47 torch.histogram and then i am plotting this histogram so basically what this is doing is that
历史图,然后我正在绘制这个直方图 所以基本上这是在做

1:27:52 every different type of layer and they all have a different color we are looking at how many
每种不同的层层 都有不同颜色 我们所看到的是多少

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:27:57 values in these tensors take on any of the values below on this axis here so the first layer is
值,在这些 发压值中,这些 以下面的值为本轴上的任何值在此此处,因此第一个层是

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:28:05 fairly saturated here at 20% so you can see that it's got tails here but then everything sort of
20%的饱和度相当饱和, 所以你可以看到它有尾巴在这里 但随后所有的东西排序

1:28:11 stabilizes and if we had more layers here it would actually just stabilize at around the
如果我们在这里多加一层的话 稳定在

1:28:15 standard deviation of about 0.65 and the saturation would be roughly 5% and the reason that this
标准差约0.65和饱和度约为5%,原因

1:28:21 stabilizes and gives us a nice distribution here is because gain is set to 5 over 3 now here this
稳定稳定,给我们一个很好的分布 这里的分布是 因为收益设定在5+3

distribution
分布 分布
数学与微积分

1:28:29 gain you see that by default we initialize with 1 over square root of fan in but then here during
获取时看到默认情况下我们先初始化 1 个以上平方根的扇扇根,然后在此

1:28:36 initialization i come in and i iterate over all the layers and if it's a linear layer i boost
初始化 我进来,我循环到所有层 如果它是线性层i 推进

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:28:40 that by the gain now we saw that one so basically if we just do not use a gain then what happens
我们看到,现在我们所看到的收益, 基本上如果我们只是不使用收益 那么会发生什么

1:28:48 if i redraw this you will see that the standard deviation is shrinking and the saturation is coming
如果我重写这个,你会看到标准偏差正在缩小 饱和度即将来临

1:28:55 to zero and basically what's happening is the first layer is you know pretty decent but then
基本上,现在发生的事情是 第一层是你知道的漂亮,但然后

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:29:01 further layers are just kind of like shrinking down to zero and it's happening slowly but it's
进一步层层的再深层 就像从零到零的缩小, 它正在缓慢地发生,但是它

1:29:06 shrinking to zero and the reason for that is when you just have a sandwich of linear layers
缩小为零,原因就在于你只有一条线性层的三明治

1:29:12 alone then a then initializing our weights in this manner we saw previously would have conserved the
以我们以前所见的方式开始我们的权重 本来会保护

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ， 则相应的特征对模型来说没有任何贡献。
神经网络基础

1:29:20 standard deviation of one but because we have this interspersed 10 h layers in there these 10
标准差之一,但因为我们有10个层的中间层

1:29:26 lh layers are squashing functions and so they take your distribution and they slightly squash it
lh 层正在压压函数, 所以它们会拿走你的分布, 然后稍微压压它

distribution
分布 分布
数学与微积分

1:29:32 and so some gain is necessary to keep expanding it to fight the squashing so it just turns out
所以,一些收益是必要的 继续扩大它以对抗压碎,所以它只是结果

1:29:41 that 503 is a good value so if we have something too small like one we saw that things will come
503是一个很好的价值 所以如果我们有 的东西太小了,我们看到了东西会来

1:29:47 towards zero but if it's something too high let's do two then here we see that well let me do
something
向零方向向零方向 但是如果太高的话 我们来两下 然后在这里我们来看看

1:29:57 a bit more extreme because so it's a bit more visible let's try three okay so we see here that
更极端一点 因为这样更明显一些 让我们试三,好,我们在这里看到

1:30:03 the saturations are trying to be way too large okay so three would create way too saturated
3个会制造出过于饱和的方式

1:30:09 activations so 503 is a good setting for a sandwich of linear layers with 10 h activations
503号启动装置是一条线性层三明治的好环境 10小时启动装置

1:30:17 and it roughly stabilizes the standard deviation at a reasonable point now honestly I have no idea
并且它大致稳定了标准偏离 在合理的点上,现在诚实的说,我不知道

1:30:24 where 503 came from in PyTorch when we were looking at the coming initialization I see empirically
那时503岁来自皮托尔奇的503岁 当我们看即将到来的初始化时 我看到的是经验

pytorch
PyTorch PyTorch
代码与工程

1:30:30 that it stabilizes this sandwich of linear and 10 h and that the saturation is in a good range
稳定线性三明治和10小时的三明治 饱和度在很宽的范围

1:30:36 but I didn't actually know if this came out of some math formula I tried searching briefly for
但我不知道这是否出自 我试图简要寻找的数学公式

1:30:41 where this comes from but I wasn't able to find anything but certainly we see that empirically
是从何而来 但我找不到任何东西 但我们确实从经验上看到了

1:30:46 these are very nice ranges our saturation is roughly 5% which is a pretty good number and
这些是非常不错的幅度 我们的饱和度约为5% 这个数字相当不错

1:30:51 this is a good setting of the gain in this context similarly we can do the exact same thing
这也是在这个背景下,收益的好环境 同样我们也能做同样的事情

1:30:56 with the gradients so here is a very same loop if it's a 10 h but instead of taking the
梯度与梯度相加, 所以这里是一个非常相同的环圈, 如果是10 h, 但不用

1:31:02 layer dot out I'm taking the grad and then I'm also showing the mean and standard deviation
分层点 我拿了研究生 然后也展示出平均值和标准偏差

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:31:07 and I'm plotting the histogram of these values and so you'll see that the gradient distribution is
我正绘制这些值的直方图, 所以你可以看到梯度分布是

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

distribution
分布 分布
数学与微积分

1:31:12 fairly reasonable and in particular what we're looking for is that all the different layers
特别是我们要找的 是所有不同的层次

1:31:16 in this sandwich has roughly the same gradient things are not shrinking or exploding so we can
三明治里有大约相同的梯度 东西不会缩小或爆炸 所以我们可以

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:31:22 for example come here and we can take a look at what happens if this gain was way too small
比如,我们来这里看看 如果收益太小会怎样

1:31:27 so this was 0.5 then you see the first of all the activations are shrinking to zero but also the
所以这是0.5,然后你可以看到 所有的启动器中的第一个 正在缩到零

1:31:34 gradients are doing something weird the gradient started out here and then now they're like
梯度正在做一些奇怪的事情 梯度从这里开始,然后现在它们就像

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:31:39 expanding out and similarly if we for example have a 2 h again so like 3 then we see that also
类似地,如果我们再有一个2小时的像3小时,我们可以看到

1:31:47 the gradients have there's some asymmetry going on where as you go into deeper and deeper layers
梯度有某种不对称 在你进入深层的地层时

1:31:51 the activations are also changing and so that's not what we want and in this case we saw that
激活也在变化 所以我们不想要这个 这个案子我们看到了

1:31:56 without the use of bachelor as we are going through right now we have to very carefully set
不需要单身汉的帮助 我们现在经历的现在 我们不得不非常小心地设置

1:32:02 those gains to get nice activations in both the forward pass and the backward pass now before we
获得这些好处,以在前面的前进通道和后退通道上获得良好的激活

forward pass
前向传播 前向传播
神经网络基础

backward pass
反向传播 反向传播
神经网络基础

1:32:07 move on to bachelor normalization I would also like to take a look at what happens when we have no
继续学士正常化,我还想看看当我们没有

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:32:12 10 h units here so erasing all the 10 h nonlinearities but keeping the gain at 5 over 3
10个单位在这里,所以删除了所有10个非线性小时的10个单位,但将收益保持在5个以上

1:32:19 we now have just a giant linear sandwich so let's see what happens to the activations
现在我们只有一个巨大的线性三明治 让我们看看激活的下场

1:32:24 as we saw before the correct gain here is one that is the standard deviation preserving gain
正如我们之前所看到的 正确的收益是标准偏差 保持收益

1:32:29 so 1.667 is too high and so what's going to happen now is the following I have to change this to be
所以1.667太高了 所以接下来我要改变这个

1:32:38 linear so we are because there's no more 10 h layers and let me change this to linear as well
线性线线,所以我们是 因为没有10 h 层了, 让我把这个也换成线性

1:32:44 so what we're seeing is the activations started out on the blue and have by layer 4 become very diffuse
启动于蓝色开始, 由4层变得非常分散。

layer ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:32:53 so what's happening to the activations is this and with the gradients on the top layer the activation
启动时发生的情况是, 并且随着顶层的梯度

layer ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

activation
激活值 激活值
神经网络基础

1:33:00 the gradient statistics are the purple and then they diminish as you go down deeper in the layers
梯度统计是紫色的,然后随着你深到层层而减少

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:33:06 and so basically you have an asymmetry like in the neural net and you might imagine that if
所以基本上,你有一个不对称 和神经网络一样的不对称

neural net
神经网络 神经网络
神经网络基础

1:33:10 you have very deep neural networks say like 50 layers or something like that this just this is not
你有非常深的神经网络 说像50层或类似的东西,这只是这

1:33:15 a good place to be so that's why before bachelor normalization this was incredibly tricky to to set
在一个好地方,所以这就是为什么 在学士正常化之前 这是难以置信的难搞

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:33:22 in particular if this is too large of a gain this happens and if it's too little of a gain
尤其是如果这太大的收益 就会发生,如果它太小的收益

1:33:28 then this happens so the opposite of that basically happens here we have a shrinking
然后,事情发生 与这相反 基本上发生 在这里我们有一个缩小

1:33:36 shrinking and a diffusion depending on which direction you look at it from
和缩小扩散取决于您看它的方向

1:33:42 and so certainly this is not what you want and in this case the correct setting of the gain is
绝不然,这不是你们所欲的;在这种情况下,利得的正确处于(和)

1:33:46 exactly one just like we're doing at initialization and then we see that the statistics for the
和我们刚开始时做的一模一样 然后我们看到

1:33:53 forward and the backward paths are well behaved and so the reason I want to show you this is
后退道路都表现良好 所以我想给你们展示这个

1:33:59 that basically like getting neuralness to train before these normalization layers
基本上就像在正常的层次之前 获得神经神经学训练

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:34:04 and before the use of advanced optimizers like Adam which we still have to cover
并且使用像亚当这样的先进优化器之前 我们仍必须覆盖

adam
Adam 优化器 Adam 优化器
训练与优化

1:34:08 and residual connections and so on training neural nuts basically look like this
和剩余连接,等等 训练神经坚果基本上看起来是这样的

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:34:13 it's like a total balancing act you have to make sure that everything is precisely orchestrated
就像一个完全平衡的动作 你必须确保一切是精确安排

1:34:18 and you have to care about the activations and the gradients and their statistics and then maybe
并且你必须关心激活和梯度 以及它们的统计,然后也许

1:34:22 you can train something but it was basically impossible to train very deep networks and this is
你可以训练一些东西 但是基本上不可能 训练非常深的网络

1:34:25 fundamentally the reason for that you'd have to be very very careful with your initialization
根本原因 为何你必须非常小心地 开始你的初始化

1:34:31 the other point here is you might be asking yourself by the way I'm not sure if I covered this
另一点是 你可能问自己 我不知我是否覆盖了这件事

1:34:37 why do we need these 10h layers at all why do we include them and then have to worry about the game
为什么我们需要这10个层 为什么要把它们都包括进去 然后还要担心游戏

1:34:43 and the reason for that of course is that if you just have a stack of linear layers
当然,原因就是 如果你只是有一堆线性层

1:34:47 then certainly we're getting very easily nice activations and so on but this is just a
我们当然会得到非常容易的 漂亮的激活装置等等,但是这只是

1:34:53 massive linear sandwich and it turns out that it collapses to a single linear layer in terms
直线三明治,结果它以单线层的值表示崩溃

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:34:57 of its representation power so if you were to plot the output as a function of the input you're
显示输出的表达能力, 所以如果您要根据输入函数绘制输出

1:35:02 just getting a linear function no matter how many linear layers you stack up you still just end up
仅仅获得线性函数,不管你堆叠多少线性层,最后还是会

1:35:07 with a linear transformation all the wx plus b's just collapse into a large wx plus b with slightly
直线转换 所有 wx+b 只是崩溃成一个大的 wx+B 稍微

1:35:14 different w's and slightly different b but interestingly even though the forward pass
虽有前方的过路,但有趣的是

forward pass
前向传播 前向传播
神经网络基础

1:35:19 collapses to just a linear layer because of back propagation and the dynamics of the backward
后向传播和后向动态导致倒向到线性层

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

back propagation
反向传播 反向传播
神经网络基础

1:35:25 pass the optimization is really is not identical you actually end up with all kinds of interesting
过牌优化的大小和您最后的 各种有趣的东西不同

1:35:32 dynamics in the backward pass because of the the way the chain rule is calculating it
后过是因为链规则在计算它的方式

backward pass
反向传播 反向传播
神经网络基础

chain rule
链式法则 链式法则
反向传播与梯度

1:35:37 and so optimizing a linear layer by itself and optimizing a sandwich of 10 linear layers
使线性层本身优化,并使10线性层的三明治最优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:35:43 in both cases those are just a linear transformation in the forward pass but the training dynamics
在这两种情况下,这些只是前通过的线性转变,但培训动态

forward pass
前向传播 前向传播
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:35:47 would be different and there's entire papers that analyze in fact like infinitely layered
将会是不同的 而且有整个文件 分析实际上像无限的层

1:35:52 linear layers and so on and so there's a lot of things too that you can play with there
线性层等等,所以有很多东西 你可以玩在那里

1:35:58 but basically the 10-inch nonlinearities allow us to
但基本上10英寸的非线性 允许我们

1:36:03 turn this sandwich from just a linear function into a neural network that can in principle
将这个三明治从一个线性函数 从一个直线函数变成一个原则上可以

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

1:36:13 approximate any arbitrary function okay so now I've reset the code to use the linear
任意性功能的大致任意性功能可以,所以现在我重新设置代码,使用线性

1:36:18 10-inch sandwich like before and I reset everything so the gains 5 out of 3 we can run a single step
10英寸三明治,像以前一样,10英寸三明治 我重置一切 所以3分5分5分

1:36:24 of optimization and we can look at the activation statistics of the forward pass and the backward
和后向的动态统计

activation
激活值 激活值
神经网络基础

forward pass
前向传播 前向传播
神经网络基础

1:36:29 pass but I've added one more plot here that I think is really important to look at when you're
但我在这里又添加了一个图, 我认为当您在

1:36:33 training your neural nets and to consider and ultimately what we're doing is we're updating
训练你的神经网 考虑最终我们所做的是更新

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:36:38 the parameters of the neural net so we care about the parameters and their values and their
神经网的参数 所以我们关心这些参数和它们的值

neural net
神经网络 神经网络
神经网络基础

1:36:43 gradients so here what I'm doing is I'm actually iterating over all the parameters available and
所以我在这里做的是 我实际上正在对所有参数进行循环

随机梯度下降 · Adam 优化器 1:36:47

1:36:47 that I'm only restricting it to the two-dimensional parameters which are basically the weights of
我将它限制在二维参数上 基本上就是

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:36:53 these linear layers and I'm skipping the biases and I'm skipping the gammas and the
这些线性层 我跳过偏向 跳过了伽马和

1:36:59 betas in the bachelor just for simplicity but you can also take a look at those as well
单身汉的贝塔只是简单的,但你也可以看一看那些

1:37:03 but what's happening with the weights is instructive by itself so here we have all
但体重本身 却有启发性 所以在这里我们都有

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:37:10 the different weights their shapes so this is the embedding layer the first linear layer all the way
不同的重量,它们的形状 所以这是嵌入层 第一个线性层

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:37:15 to the very last linear layer and then we have the mean the standard deviation of all these parameters
直线到最后一层,然后我们有所有参数的标准偏差平均值

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:37:21 the histogram and you can see that it actually doesn't look that amazing so there's some trouble
直方图和你可以看到 它其实看起来并不那么惊人 所以有些麻烦

1:37:25 in paradise even though these gradients looked okay there's something weird going on here
天堂里,即使这些梯度看起来还不错 这里也有些奇怪

1:37:30 I'll get to that in a second and the last thing here is the gradient to data ratio so sometimes
我马上去问这个,这里最后一件事 就是梯度对数据比率 所以有时

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:37:36 I like to visualize this as well because what this gives you a sense of is what is the scale of the
我想想象一下这个,因为 是什么让你有感觉的 是什么是规模

1:37:41 gradient compared to the scale of the actual values and this is important because we're going to end
相对于实际值的尺度, 渐变与实际值的大小比较, 这很重要, 因为我们要结束

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:37:47 up taking a step update that is the learning rate times the gradient onto the data and so the
以一个步骤更新,即学习率乘数据上的梯度乘以该梯度,因此

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:37:54 gradient has too large of a magnitude if the numbers and there are too large compared to the
如果数字与数值相比大,则梯度的大小太大,如果数字和与

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:37:59 numbers and data then you'd be in trouble but in this case the gradient to data is our
数字和数据 然后你就会有麻烦 但在这种情况下,数据梯度是我们的

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:38:04 low numbers so the values inside grad are 1000 times smaller than the values inside data in these
低数值,所以在梯度中的值比这些数值中的数据的值小1000倍

1:38:11 weights most of them now notably that is not true about the last layer and so the last layer
大部分的值正负数, 现在明显地是,关于最后一层, 也就是最后一个层, 并不是这样

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:38:17 actually here the output layer is a bit of a troublemaker in the way that this is currently
这里的输出层实际上 是一个有点麻烦的制造者, 因为它是目前的方式

output layer
输出层 神经网络的 " 最后 " 一层，也是包含答案的层。
神经网络基础

1:38:21 arranged because you can see that the last layer here in pink takes on values that are
粉红色中最后一层的值

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:38:29 much larger than some of the values inside the neural net so the standard deviations are roughly
大大大于神经网内的某些值,因此标准偏差大致是

neural net
神经网络 神经网络
神经网络基础

1:38:37 wanting negative three throughout except for the last last new layer which actually has roughly
除了最后一层新层,实际上大致上是

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:38:42 wanting negative two standard deviation on gradients and so the gradients on the last layer are
想要在梯度上以负差为两个标准差,所以最后一层的梯度是

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:38:47 currently about 100 times greater sorry 10 times greater than all the other weights inside
目前大约100倍以上 少于里面所有其它重量的10倍

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:38:54 the neural net and so that's problematic because in the simple stochastic gradient in the send setup
神经网,所以这是个问题 因为发送装置中的简单随机梯度

neural net
神经网络 神经网络
神经网络基础

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:39:00 you would be training this last layer about 10 times faster than you would be training the other
训练最后一层的你比训练另一层要快10倍

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:39:04 layers at initialization now this actually like kind of fixes itself a little bit if you train
初始化时的层, 现在它实际上就像某种修整本身一样, 如果您训练

1:39:10 for a bit longer so for example if I grew here then 1000 only then do a break let me
更长一点,例如,如果我在这里长大,那么1000只到这里,然后休息一段时间,让我休息

1:39:16 initialize and then let me do it 1000 steps and after 1000 steps we can look at the forward
初始化, 然后让我做1000步, 在1000步之后, 我们可以看前面

1:39:23 pass okay so you see how the neurons are a bit are saturating a bit and we can also look at the
就可以看看神经元的 饱和度

1:39:28 backward pass but otherwise they look good they're about equal and there's no shrinking to zero or
后向过,但其它情况看起来不错,它们几乎相等,没有缩到零或

backward pass
反向传播 反向传播
神经网络基础

1:39:34 exploding to infinities and you can see that here in the weights things are also stabilizing a
爆炸到无限的无无无无不的宇宙中, 你可以看到在重量中,

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:39:39 little bit so the tails of the last pink layer are actually coming coming in during the optimization
粉红色末层的尾巴 在优化时会来

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:39:46 but certainly this is like a little bit of troubling especially if you are using a very
但确实有点麻烦 尤其是如果你使用

1:39:50 simple update rule like stochastic gradient descent instead of a modern optimizer like Adam
简单更新规则, 如随机梯度下坡, 而不是像亚当这样的现代优化器

stochastic gradient descent
随机梯度下降 随机梯度下降
反向传播与梯度

adam
Adam 优化器 Adam 优化器
训练与优化

1:39:55 now I'd like to show you one more plot that I usually look at when I train neural networks and
现在我想给你们展示一个情节, 当我训练神经网络时我通常会看到这个情节。

1:39:59 basically the gradient to data ratio is not actually that informative because what matters
基本来说,渐渐度对数据比率实际上不是信息性的数据比,因为重要的是什么

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:40:03 at the end is not the gradient to data ratio but the update to the data ratio because that
结尾处不是梯度对数据比率,而是数据比率的更新,因为

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:40:08 is the amount by which we will actually change the data in these tensors so coming up here
是我们实际改变这些 数组数据的数量, 所以从这里上传到这里

1:40:14 what I'd like to do is I'd like to introduce a new update to data ratio it's going to
我想做的是,我想介绍 新的数据比率更新

1:40:19 be less than we're going to build it out every single iteration and here I'd like to keep track of
少于我们建造它 每一个循环,在这里 我想保持跟踪

iteration [.itə'reiʃən]
迭代 模型的权重在训练期间的一次更新。迭代包含计算参数在单批次 数据上的梯度损失。 K k-means 一种热门的聚类 算法，用于对非监督式学习中的样本进行分组。 k-means 算法基 本上会执行以下操作： 以迭代方式确定最佳的 k 中心点（称为形心 ）。 将每个样本分配到最近的形心。与同一个形心距离最近的样本属于同一个组。 k-means
神经网络基础

1:40:24 basically the ratio every single iteration so without any gradients I'm comparing the update
基本上每个迭代的比 所以没有梯度 我比较更新

iteration [.itə'reiʃən]
迭代 模型的权重在训练期间的一次更新。迭代包含计算参数在单批次 数据上的梯度损失。 K k-means 一种热门的聚类 算法，用于对非监督式学习中的样本进行分组。 k-means 算法基 本上会执行以下操作： 以迭代方式确定最佳的 k 中心点（称为形心 ）。 将每个样本分配到最近的形心。与同一个形心距离最近的样本属于同一个组。 k-means
神经网络基础

1:40:34 which is learning rate times the times the gradient that is the update that we're going to
学习率是学习率乘以梯度的倍数, 也就是我们将要更新的梯度

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:40:39 apply to every parameter so see I'm iterating over all the parameters and then I'm taking the
应用到每个参数时, 每一个参数都应用到所有参数上, 这样我可以看到, 我在所有参数上反复循环, 然后我就拿了

parameter [pə'ræmitə]
参数 机器学习系统自行训练的模型的变量。例如，权重 就是一种参数，它们的值是机器 学习系统通过连续的训练迭代逐渐学习到的。与超参数 相对。
神经网络基础

1:40:45 basically standard deviation of the update we're going to apply and divided by the actual content
基本上标准差 更新我们将应用 并除以实际内容

1:40:52 the data of that parameter and its standard deviation so this is the ratio of basically
该参数的数据及其标准偏差,因此这是基数的比

parameter [pə'ræmitə]
参数 机器学习系统自行训练的模型的变量。例如，权重 就是一种参数，它们的值是机器 学习系统通过连续的训练迭代逐渐学习到的。与超参数 相对。
神经网络基础

1:40:58 how great are the updates to the values in these tensors then we're going to take a log of it
这些变压器中值的更新 有多棒? 然后我们用它做一个日志

1:41:03 and actually I'd like to take a log 10 just so it's a nicer visualization so we're going to
实际上我想拿一个原10日的正数 仅仅这样它更美一些视觉化 所以我们要去

1:41:10 be basically looking at the exponents of the of this division here and then that item to pop out
基本上就是看这个部门 这里的推手 然后那个东西就弹出来了

1:41:18 the float and we're going to be keeping track of this for all the parameters and adding it to these
并加入这些参数

1:41:22 ud tensor so now let me re-initialize and run a thousand iterations we can look at the
都变了,现在让我重新开动 并运行一千次我们可以看到

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:41:29 activations the gradients and the parameter gradients as we did before but now I have one
激活梯度和参数梯度,就像我们以前一样,但现在我有一个

parameter [pə'ræmitə]
参数 机器学习系统自行训练的模型的变量。例如，权重 就是一种参数，它们的值是机器 学习系统通过连续的训练迭代逐渐学习到的。与超参数 相对。
神经网络基础

1:41:34 more plot here to introduce and what's happening here is where every interval to parameters
在此更多要介绍的绘图和这里发生的事情是参数的间隔

1:41:40 and I'm constraining it again like I did here to just the weights so the number of dimensions in
我再次约束它 就像我在这里所做的一样 仅仅重量,所以数量 维度

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:41:46 these sensors is two and then I'm basically plotting all of these update ratios over time
这些传感器是两个 然后基本上我正在 绘制这些更新率的图

1:41:54 so when I plot this I plot those ratios and you can see that they evolve over time during
所以当我绘制这个图时,我绘制了这些比率, 你可以看到它们随着时间的演变

1:41:59 initialization to take on certain values and then these updates sort of like start
初始化, 以获取某些值, 然后这些更新类似开始

1:42:03 stabilizing usually during training then the other thing that I'm plotting here is I'm
训练期间通常稳定下来... ...然后我在这里想的另一个事情是...

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:42:07 plotting here like an approximate value that is a rough guide for what it roughly should be
在这里像一个大致值一样 绘制一个粗略的指南, 说明它应该是什么

1:42:12 and it should be like roughly 1 in negative 3 and so that means that basically there's some values in
应该是大约1的负3, 所以这意味着基本上有一些价值

1:42:18 this tensor and they take on certain values and the updates to them at every single iteration
并采用某些价值观 并在每一循环中更新它们

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

iteration [.itə'reiʃən]
迭代 模型的权重在训练期间的一次更新。迭代包含计算参数在单批次 数据上的梯度损失。 K k-means 一种热门的聚类 算法，用于对非监督式学习中的样本进行分组。 k-means 算法基 本上会执行以下操作： 以迭代方式确定最佳的 k 中心点（称为形心 ）。 将每个样本分配到最近的形心。与同一个形心距离最近的样本属于同一个组。 k-means
神经网络基础

1:42:24 are no more than roughly one thousandth of the actual like magnitude in those tensors if this
不超过这些数组中实际类似数量之千分之一的数值,如果是的话。

1:42:31 was much larger like for example if this was if the log of this was like say negative one this is
更像,比如说如果这个日志是... ...就像说负的

1:42:37 actually updating those values quite a lot they're undergoing a lot of change but the reason that the
实际上,这些价值的更新 相当很多 它们正在经历着许多变化 但原因

1:42:43 final rate the final layer here is an outlier is because this layer was artificially shrunk down
最后的电流率最后的层是外值, 是因为这个层人为缩小了

layer ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:42:50 to keep the softmax incompetent so here you see how we multiply the weight by point one
使软负不力,所以在这里你可以看到我们如何将重量乘以一

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:42:59 in the initialization to make the last layer prediction less confident that made that artificially
使最后一层预测 降低信心

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:43:06 made the values inside that tensor way too low and that's why we're getting temporarily a very high
使得内部的值值 以高压的方式太低 所以我们暂时变得非常高

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:43:11 ratio but you see that that stabilizes over time once that weight starts to learn starts to learn
体重开始学习后,就会逐渐稳定下来

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:43:17 but basically I like to look at the evolution of this update ratio for all my parameters
但我基本上喜欢看看 我所有参数的更新率的变化

1:43:22 usually and I like to make sure that it's not too much above 1 in negative 3 roughly
通常我也要确保它不会超过1, 大约是负3

1:43:29 so around negative 3 on this log plot if it's below negative 3 usually that means that the
通常意味着

1:43:34 parameters are not trained fast enough so if our learning rate was very low let's do that
训练不够快,所以如果我们的学习率很低的话 我们来做吧

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:43:39 experiment let's initialize and then let's actually do a learning rate of say 1 in negative 3
实验让我们开始开始,然后我们实际做一个学习率, 以负3表示 1

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:43:46 here so 0.001 if your learning rate is way too low this plot will typically reveal it
所以这里是0.001 如果你的学习率太低的话 这个情节通常会暴露出来

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:43:56 so you see how all of these updates are way too small so the size of the update is basically
所以,你可以看到所有这些更新的大小 都太小了,所以更新的规模基本上就是

1:44:03 10 000 times in magnitude to the size of the numbers in that tensor in the first place
以10 000倍的大小表示,数量大小在当年的时速

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:44:10 so this is a symptom of training way too slow so this is another way to sometimes
所以这是训练速度太慢的症状,所以 这是另一种方法 有时

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:44:15 set the learning rate and to get a sense of what that learning rate should be and ultimately
确定学习率,并了解学习率应该和最终应该是什么

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:44:19 this is something that you would keep track of if anything the learning rate here is a little bit
这一点你一定能追踪到 如果这里的学习率有一点点

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:44:27 on the higher side because you see that we're above the black line of negative 3 we're somewhere
上面的一面,因为你看我们 越过了3号负3号黑线 我们到了某个地方

1:44:34 around negative 2.5 it's like okay and but everything is like somewhat stabilizing and so this looks
大约负2.5度,它像是正常的,但一切都像稳定下来 所以这看起来

1:44:40 like a pretty decent setting of of learning rates and so on but this is something to look at
学习率的相当不错,但是这是值得一看的

1:44:45 and when things are miscalibrated you will you will see very quickly so for example
当事情被错误地调整的时候,你们将看见许多瞬间,例如

1:44:50 everything looks pretty well behaved right but just as a comparison when things are not properly
一切都看起来都很好 表现不错,但只是比较 当事情不正确时

1:44:54 calibrated what does that look like let me come up here and let's say that for example
校准了那看起来像什么 让我上来,比如说

1:45:00 what do we do let's say that we forgot to apply this fan-in normalization so the weights
我们该说什么呢? 说我们忘了应用这种"迷恋"的正常化,所以权重

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:45:06 inside the linear layers are just a sample from a Gaussian in all the stages what happens to
线性层内层只是高森人的样本 在所有阶段发生的事情

sample
样本 样本
神经网络基础

1:45:12 how do we notice that something's off well the activation plot will tell you whoa your
我们如何注意到 东西的关闭 井井有灵 启动计划会告诉你

activation
激活值 激活值
神经网络基础

1:45:16 neurons are way too saturated the gradients are going to be all messed up the histogram for these
梯度会完全混乱地 对这些神经元的直方图

1:45:22 weights are going to be all messed up as well and there's a lot of asymmetry and then if we
体重也会被弄乱 而且还存在许多不对称

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:45:27 look here I suspect it's all going to be also pretty messed up so you see there's a lot of
我怀疑一切都会很混乱 所以你可以看到有很多

1:45:32 discrepancy in how fast these layers are learning and some of them are learning way too fast so
这些层层学习速度的差异,有些层学习速度过快。

1:45:39 negative 1 negative 1.5 those are very large numbers in terms of this ratio again you should be somewhere
负1负1.5,从这个比例来看,这些数字非常大 你该再去某个地方

1:45:45 around negative 3 and not much more above that so this is how miscalibration of your neural
大约3个负数,仅比这个低得多 所以这就是如何错误校正你的神经

1:45:51 nodes are going to manifest and these kinds of plots here are a good way of sort of bringing
节点会显示,这些图案是一个很好的方式

1:45:58 those miscalibrations sort of to your attention and so you can address them okay so so far
让你注意那些错误的校正 以便你解决他们 目前为止还行

1:46:05 we've seen that when we have this linear 10h sandwich we can actually precisely calibrate the
我们见过 当我们有了这个线性10小时三明治时, 我们可以精确校准

损失函数 · 学习速率 1:46:09

1:46:09 gains and make the activations the gradients and the parameters and the updates all look pretty decent
使启动的梯度、参数和更新看起来都非常体面

1:46:15 but it definitely feels a little bit like balancing of a pencil on your finger and that's because
但感觉肯定有点像 平衡你手指上的铅笔

1:46:22 this gain has to be very precisely calibrated so now let's introduce vaturalization layers into
得精确校准这个收益 所以现在让我们把 立体化层引入

1:46:27 the fix into the mix and let's let's see how that helps fix the problem so here i'm going to take the
来,让我们来看看这如何帮助解决问题 所以我要在这里采取

1:46:36 bachelor 1d class and i'm going to start placing it inside and as i mentioned before the standard
学士一等,我准备开始放进去 就像我在标准之前提到的那样

1:46:43 typical place you would place it is between the linear layer so right after it but before the
通常的位置位置,您会把它放在线性层之间。 在线性层之后,在

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:46:48 non-linearity but people have definitely played with that and in fact you can get very similar
与非线性无线性 但人们肯定玩过这个游戏 事实上你也可以变得非常相似

1:46:53 results even if you place it after the non-linearity and the other thing that i wanted to mention is
即使你把它放在非线性之后 和我想提的另外一件事

1:46:59 it's totally fine to also place it at the end after the last linear layer and before the
也完全可以放在最后线性层的结尾处, 在

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:47:03 loss function so this is potentially fine as well and in this case this would be output
功能损失,因此这也可能是罚款,在这种情况下,这是产出

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:47:11 would be vocab size now because the last layer is bachelor we would not be changing the weight
将变成现在的葡萄球体大小 因为最后一层是单身 我们不会改变体重

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:47:18 to make the softmax less confident we'd be changing the gamma because gamma remember in the
使软最大更不自信 我们将会改变伽玛 因为伽马在

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:47:24 bachelor is the variable that multiplicatively interacts with the output of that normalization
学士是可倍增地与正常化输出互动的变量

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:47:32 so we can initialize this sandwich now we can train and we can see that the activations
这样我们可以开始这个三明治的初始化 现在我们可以训练,我们可以看到 激活

1:47:39 are going to of course look very good and they are going to necessarily look good because now
当然会看起来非常漂亮, 并且它们一定会看起来很好 因为现在

1:47:44 before every single 10h layer there is a normalization in the bachelor so this is unsurprisingly
10层之前的每个10层 单身汉都有一个正常化 所以这不奇怪

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:47:51 all looks pretty good it's going to be standard deviation of roughly 0.65 2% and roughly equals
看上去都不错,这是标准差 大约0.652%,大致相等

1:47:57 standard deviation throughout the entire layers so everything looks very homogeneous the gradients
整个层的标准差,所以所有各层看起来都非常均匀

1:48:03 look good the weights look good and their distributions and then the updates also look
重量看起来不错,重量看上去很好 分布和更新也一样好看

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:48:12 pretty reasonable we're going above negative 3 a little bit but not by too much so all the
相当合理,我们正向负3 略微超过负3,但并不太多

1:48:18 parameters are training at roughly the same rate here but now what we've gained is we are going to
参数训练率与这里的大致相同 但现在我们得到的是

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:48:26 be slightly less brittle with respect to the gain of these so for example i can make the gain be
相对于这些收益的利得,我略微少一点易碎,例如我可以使收益

1:48:35 say 0.2 here which is much as much slower than what we had with the 10h but as we'll see the
说0.2 这里比我们10小时的慢得多 但是我们会看到

1:48:43 activations will actually be exactly unaffected and that's because of again this explicit normalization
启动装置将完全不受影响 因为这种明确的正常化

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:48:49 the gradients are going to look okay the weight gradients are going to look okay but actually the
渐变看起来不错 重量梯度看上去不错 但实际上

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ， 则相应的特征对模型来说没有任何贡献。
神经网络基础

1:48:54 updates will change and so even though the forward and backward paths to a very large extent look
更新将会改变,所以即使前向和后向路径的外观会大为改观

1:49:01 okay because of the backward paths of the bachelor and how the scale of the incoming
是因为单身汉的后退路 以及新进者的规模

1:49:05 activations interacts in the bachelor and its backward paths this is actually changing
激活在单身汉的后退路径中互动 这实际上正在改变

1:49:11 the um the scale of the updates on these parameters so the gradients of these weights
和这些参数的更新比例,因此这些加权值的梯度

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ， 则相应的特征对模型来说没有任何贡献。
神经网络基础

1:49:18 are affected so we still don't get a completely free pass to pass in arbitrary um weights here
受到影响 所以我们还是不能完全自由通过 任意的重量

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ， 则相应的特征对模型来说没有任何贡献。
神经网络基础

1:49:24 but everything else is significantly more robust in terms of the forward backward
但其他一切 都比前向后方强得多

1:49:31 and the weight gradients it's just that you may have to retune your learning rate
和重量梯度,只是你可能需要调整你的学习率

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:49:35 if you are changing sufficiently the um the scale of the activations that are coming into the
如果您在充分修改 um 启动的大小

1:49:40 bachelor's so here for example this um we changed the gains of these linear layers to be greater
单身汉在这里,比如这个... 我们改变了这些线性层的收益

1:49:47 and we're seeing that the updates are coming out lower as a result and then finally we can also
我们看到最新消息 正在下降 作为结果,然后我们最终也可以

1:49:53 if we are using bachelor's we don't actually need to necessarily let me reset this to one so there's
如果我们用单身汉的学士的 我们其实不需要让我把这个重置到一个

1:49:58 no gain we don't necessarily even have to um normalize by fan in sometimes so if I take out
连我们不一定得分 也不一定非得平时通过扇风来正常化 有时这样的话 如果我干掉

1:50:04 the fan in so these are just now uh random Gaussian we'll see that because of bachelor this will
粉丝在,所以这些是... ... 呃随机高森 我们可以看到,因为单身汉

1:50:09 actually be relatively well behaved so this this is look of course in the forward pass look good
表现得还不错,所以这当然是前路的外观,看起来不错。

forward pass
前向传播 前向传播
神经网络基础

1:50:17 the gradients look good the uh backward uh weight updates look okay a little bit of fat tails on some
粗体后脑重量更新看起来不错 有一些是脂肪尾巴

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:50:25 of the layers and uh this looks okay as well but as you as you can see uh we're significantly
和呃这看起来不错,但你可以看到, 嗯我们很显著

1:50:32 below negative three so we'd have to bump up the learning rate of this bachelor uh so that
低于负3,所以我们不得不提高 这个单身汉的学习率 呃,这样

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:50:37 we are training more properly and in particular looking at this roughly looks like we have to
我们训练得更恰当 尤其要看这个大概看来

training ['treinɪŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:50:41 10x the learning rate to get to about 1-3 so we've come here and we would change this to be
10x的学习率,达到约1 -3, 所以我们来到这里,我们会改变这个

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:50:49 update of 1.0 and if I initialize
更新 1. 0 和如果我初始化

1:50:59 then we'll see that everything still of course looks good and now we are roughly here and we
然后我们就会看到一切 仍然看起来还不错 现在,我们大概在这里

1:51:04 expect this to be an okay training run so long story short we are significantly more robust to
期待这能成为一次训练,

training ['treinɪŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:51:10 the gain of these linear layers whether or not we have to apply the fan in and then we can change
这些线性层的增益, 是否我们必须应用扇子来加入, 然后我们可以更改

1:51:15 the gain but we actually do have to worry a little bit about the update scales and making sure that
增加的量值,但实际上我们确实需要担心 更新比例表,确保

1:51:21 the learning rate is properly calibrated here but the activations of the forward backward pass and
学习率在此进行了适当校准,但前后向通行证和

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

backward pass
反向传播 反向传播
神经网络基础

1:51:26 the updates are all are looking significantly more well behaved except for the global scale
除了全球规模外,更新情况都看起来表现良好得多

1:51:31 that is potentially being adjusted here okay so now let me summarize there are three things I
有可能调整 这里,好吗 所以现在让我总结一下 有三件事我

1:51:37 was hoping to achieve with this section number one I wanted to introduce you to bachelor normalization
希望这篇第一篇能实现 让你了解单身汉的正常化

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:51:42 which is one of the first modern innovations that we're looking into that helped stabilize
这也是我们研究的第一批现代创新之一 有助于稳定

1:51:46 very deep neural networks and their training and I hope you understand how the bachelor
深深的神经网络和训练 我希望你明白单身汉

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:51:51 normalization works and how it would be used in a neural network number two I was hoping to
正常的工程,如何在第二号神经网络里使用

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层 ），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

PyTorch · 反向传播 1:51:57

1:51:57 pytorchify some of our code and wrap it up into these modules so like linear bachelor 1d 10
将我们的一些代码烧成灰 并包成这些模块 就像线性单身1D10

1:52:04 age etc these are layers or modules and they can be stacked up into neural nets like lego building
等这些年龄的大小,是层或模块,可以堆叠成神经网,如连网建筑

1:52:09 blocks and these layers actually exist in pytorch and if you import torch and then you can actually
和这些层的块,这些层实际上存在于 Pytoch 中,如果你导入了火炬,然后你就可以

pytorch
PyTorch PyTorch
代码与工程

1:52:17 the way I've constructed it you can simply just use pytorch by prepending and and dot
像我建的一样 你可以简单地用火炉 预先准备和点

pytorch
PyTorch PyTorch
代码与工程

1:52:22 to all these different layers and actually everything will just work because the API that I've
和所有不同的层次 实际上一切都会工作,因为我曾经的API

1:52:28 developed here is identical to the API that pytorch uses and the implementation also is basically as
这里所制定的API与Pytor使用和 实施基本上相同

pytorch
PyTorch PyTorch
代码与工程

implementation
实现 实现
代码与工程

1:52:34 far as I'm aware identical to the one in pytorch and number three I try to introduce you to the
和《火枪》里那个一样 第三个 我试着向你介绍

pytorch
PyTorch PyTorch
代码与工程

1:52:40 diagnostic tools that you would use to understand whether your neural network is in a good state
用于了解神经网络是否处于良好状态的诊断工具

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

1:52:45 dynamically so we are looking at the statistics and histograms and activation of the forward
动态,所以我们正研究统计和直方图 并激活前方

activation
激活值 激活值
神经网络基础

1:52:50 pass after activation activations the backward pass gradients and then also we're looking at the
激活后过继启动后, 后过后梯度 并且我们也在查看

activation
激活值 激活值
神经网络基础

backward pass
反向传播 反向传播
神经网络基础

1:52:55 weights that are going to be updated as part of stochastic gradient ascent and we're looking at
将更新的重量 作为随机梯度升温的一部分 我们正研究

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:52:59 their means standard deviations and also the ratio of gradients to data or even better the updates
表示标准差,以及梯度对数据或更佳更新的数据之比

1:53:06 to data and we saw that typically we don't actually look at it as a single snapshot frozen in
数据,我们看到通常我们并不把它看成

1:53:11 time at some particular iteration typically people look at this as over time just like I've
在某些特定时间的循环中 人们通常会像我一样,视此为随时间推移

iteration [.itə'reiʃən]
迭代 模型的权重在训练期间的一次更新。迭代包含计算参数在单批次 数据上的梯度损失。 K k-means 一种热门的聚类 算法，用于对非监督式学习中的样本进行分组。 k-means 算法基 本上会执行以下操作： 以迭代方式确定最佳的 k 中心点（称为形心 ）。 将每个样本分配到最近的形心。与同一个形心距离最近的样本属于同一个组。 k-means
神经网络基础

1:53:16 done here and they look at these update to data ratios and they make sure everything looks okay
并且他们看这些更新数据比率的更新, 确保一切看起来都没事

1:53:21 and in particular I said that one in negative three or basically negative three on the lock scale
特别是我说过,在锁定比例表上,每负三分之一或基本上负三分一

1:53:27 is a good rough heuristic for what you want this ratio to be and if it's way too high then probably
是一个好粗粗的 疲劳的理论 想要这个比例是什么样的 如果它太高的话

1:53:33 the learning rate or the updates are a little too too big and if it's way too small that the
学习率或更新率都太高了 如果太小的话

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:53:37 learning rate is probably too small so that's just some of the things that you may want to
学习率可能太小 所以这只是你可能想要的

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:53:41 play with when you try to get your neural network to work with very well now there's
当你试图让神经网络 与现在的

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层 ），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

1:53:47 a number of things I did not try to achieve I did not try to beat our previous performance as an
我并没有试图打破我们以前作为一名

1:53:51 example by introducing the bachelor layer actually I did try and I found that you I used the learning
举个例子,通过介绍学士级 其实我试过尝试 我发现你我用过

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:53:57 rate finding mechanism that I've described before I tried to train the bachelor layer a
率的发现机制 我之前描述过

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:54:01 bachelor neural net and I actually ended up with results that are very very similar to what
和我的神经网结结结果非常相似

neural net
神经网络 神经网络
神经网络基础

1:54:06 we've obtained before and that's because our performance now is not bottlenecked by the
因为我们的表演现在没有被

1:54:11 optimization which is what bachelor is helping with the performance at the stage is bottlenecked
也就是单身汉在舞台表演时 帮助的优化 是瓶颈

1:54:16 by what I suspect is the context length of our context so currently we are taking three
依我猜想,我们的背景关系长度 估计是三

1:54:23 characters to predict the fourth one and I think we need to go beyond that and we need to look at
来预测第四个字的字符 并且我认为我们需要超越这个, 我们需要看

1:54:26 more powerful architectures like recurrent neural networks and transformers in order to further
更强大的建筑,如经常性神经网络和变压器,以便进一步

1:54:31 push the like probabilities that we're achieving on this data set and I also did not try to have
推推,像概率 我们所实现的概率 我们在这个数据集上实现 我也没有尝试拥有

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

1:54:38 a full explanation of all of these activations the gradients and the backward pass and the
充分解释所有这些激活梯度、后向通行证和

backward pass
反向传播 反向传播
神经网络基础

1:54:43 statistics of all these gradients and so you may have found some of the parts here unintuitive
这些数据的统计, 包括所有这些梯度, 所以您可能发现有些部分在这里不直观

1:54:47 and maybe you're slightly confused about okay if I change the gain here how come that we need a
也许你有点困惑 如果我改变收益,你是否还好

1:54:52 different learning rate and I didn't go into the full detail because you'd have to actually look
不同的学习率和我没有详细讲完 因为你必须去

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。 学习速率是一个重要的超参数 。
训练与优化

1:54:56 at the backward pass of all these different layers and get an intuitive understanding of how
并直觉了解

backward pass
反向传播 反向传播
神经网络基础

1:55:00 that works and I did not go into that in this lecture the purpose really was just to introduce
工作,我并没有去 谈到这一点 在这个讲座里,真正的目的 其实是为了介绍

1:55:05 you to the diagnostic tools and what they look like but there's still a lot of work remaining
诊断工具,以及它们长什么样, 但是还有很多工作要做

1:55:09 on the intuitive level to understand the initialization the backward pass and how
直觉上了解后向通道初始化以及如何

backward pass
反向传播 反向传播
神经网络基础

反向传播 1:55:14

1:55:14 all of that interacts but you shouldn't feel too bad because honestly we are getting to the cutting
所有的相互作用 但你不应该觉得太难过 因为说实话,我们得到的切

1:55:20 edge of where the field is we certainly have an I would say solved initialization and we haven't
场的边缘 我们肯定有一个我说解决了初始化,我们还没有

1:55:26 solved back propagation and these are still very much an active area of research people are
解后传播 而这些仍然是研究人员的一个非常活跃的领域

back propagation
反向传播 反向传播
神经网络基础

1:55:31 still trying to figure out what is the best way to initialize these networks what is the best
仍在寻找什么是启动这些网络的最佳方法 是什么才是最好的

1:55:34 update rule to use and so on so none of this is really solved and we don't really have all the
使用时更新规则等,所以这些都解决不了了 我们没有全部的

1:55:40 answers to all the to you know all these cases but at least you know we're making progress and at
答案,所有你了解 所有这些案例 但至少你知道 我们正在取得进展

1:55:46 least we have some tools to tell us whether or not things are on the right track for now
至少我们有一些工具来告诉我们 情况是否在正确的轨道上

1:55:51 so I think we've made positive progress in this lecture and I hope you enjoyed that and I
所以我认为我们在这个讲座上取得了积极的进展 我希望你和我享受到这个

1:55:56 will see you next time
下次再见

点击任意句子从该处播放 · 空格 播放/暂停 · T 中文 · C 连播 · ↑ ↓ 上下句 · Esc 停止
字幕由 ASR 转录并经 AI 校正重译，术语释义仅供参考。